

Given this new information on differences, what should be my implementation strategy. Which libraries to use and what are the next steps to start implemetnation

Thought for 11 seco... >

Summary:

To rapidly prototype a Restless Multi-Armed Bandit (RMAB) scheduler for Suvita, start with a lightweight bandit library—like **bayesianbandits**, **MABWiser**, or **SMPyBandits**—that you can wrap in a **FastAPI** microservice exposing `/infer` and `/recommend` endpoints. Containerize with **Docker** and deploy on a serverless platform (e.g., Google Cloud Functions or AWS Lambda). Integrate outbound SMS/call actions via **Twilio**. Phased steps ensure you deliver a working MVP in 4–6 weeks, prove engagement gains, then iterate with more advanced RMAB policies or context features.

1. Library Selection & Rationale

1.1 bayesianbandits (✓ Restless Support)

A simple Python package built atop scikit-learn for **Bayesian multi-armed bandits**, offering fast inference via conjugate priors and easy API (`Agent.learn()` / `Agent.recommend()`)—ideal for RMAB prototyping and supports delayed rewards contexts [GitHub](#) .

1.2 MABWiser (✓ Contextual & Prototyping)

Fidelity’s scikit-learn–style bandit library for **context-free & contextual** policies, with built-in parallel simulation and hyperparameter tuning. Requires only `pip install mabwiser`, then a few lines to `fit()` and `predict()` on your historical logs [fidelity.github.io](#) [GitHub](#) .

1.3 SMPyBandits (✓ Classical Algorithms & UCBoost)

A pure-Python package offering **65+ classic and UCBoost** policies—including UCB(d)-based indices—via an intuitive interface. Great for benchmarking simple Whittle-index approximations and UCBoost’s constant-time index updates [GitHub](#) [SMPyBandits](#) .

1.4 markovianbandit-pkg (✓ Exact Whittle Indices)

If you need exact Whittle-index computation for finite-state Markovian bandits, this PyPI package computes indices and tests indexability out-of-the-box—useful for comparing approximate policies against the true optimum [PyPI](#) .

2. Implementation Strategy

2.1 Phase 1: Proof of Concept (Weeks 1–2)

- 1. **Dummy Data:** Generate ~100 synthetic beneficiary rows (ID, state, last contact, outcome) to simulate engagement transitions.
- 2. **Bandit Service:** Choose **bayesianbandits** or **MABWiser** and write a small Python script:

```
from bayesianbandits import Agent agent = Agent(arms=['B1', 'B2', ...])
agent.learn(history) # `/infer` recs = agent.recommend(k=5) # `/recommend`
```

- 3. **FastAPI Wrapper:** Expose two endpoints using FastAPI—one for parameter inference and one for recommendation—leveraging FastAPI’s rapid model-serving pattern [Medium](#) .

2.2 Phase 2: Integration & Deployment (Weeks 3–4)

- 1. **Containerization:** Write a `Dockerfile` based on Python slim, install dependencies, and copy your FastAPI app—follow Tiangolo’s Docker guide for FastAPI [FastAPI](#) .
- 2. **Serverless Deployment:** Deploy the container to Google Cloud Run or adapt with Mangum for AWS Lambda/Azure Functions [Reddit](#) .
- 3. **SMS/Call Write-Back:** Use Twilio’s Python SDK to send outbound messages in your `/recommend` handler:

```
from twilio.rest import Client client = Client(ssid, auth_token)
client.messages.create(to=phone, from_=twilio_num, body=msg) ```
:contentReference[oaicite:9]{index=9}:contentReference[oaicite:10]{index=10}.
```

2.3 Phase 3: Monitoring & Iteration (Weeks 5–6)

- 1. **Logging & Metrics:** Instrument endpoints with Prometheus or Cloud Logging to track request rates, latencies, and success/failure counts.
- 2. **A/B Testing:** Route a portion of traffic to your bandit service and compare engagement rates vs. heuristic control.
- 3. **Hyperparameter Tuning:** Use MABWiser’s built-in simulation utilities to optimize exploration parameters offline [fidelity.github.io](#) .

3. Next Steps

- 1. **Data Ingestion:** Coordinate with Suvita to obtain:
 - Beneficiary register (demographics, IDs)
 - Historical SMS/call logs (timestamps, outcomes)
 - Clinic attendance records (engaged vs. dropped)
- 2. **Schema Definition:** Finalize JSON schemas for `/infer` (batch history) and `/recommend` (current states) to abstract away ETL details.

3. **Prototype Build:** Stand up a local FastAPI service, demo with dummy data, and validate your endpoints.
4. **Pilot Deployment:** Deploy to staging, integrate with one regional ambassador team, and measure click-through and attendance lift.
5. **Scale & Enhance:** Introduce contextual features (e.g., district, gestational week) with MABWiser; benchmark against SMPyBandits UCBBoost and markovianbandit-pkg indices; plan production rollout.

By following this **phased strategy**—starting simple with **bayesianbandits** or **MABWiser**, wrapping in **FastAPI**, and integrating **Twilio** for write-back—you'll have a working RMAB MVP within **4–6 weeks**. From there, you can incrementally adopt more complex policies or contextual modules as data and resources permit.

how to decide between bayesianbandits or MABWiser

Thought for 12 seco... >

Summary

Choosing between **bayesianbandits** and **MABWiser** hinges on three core considerations: (1) **feature support**—does your MVP need restless/delayed rewards or contextual modeling out of the box? (2) **API style & prototyping speed**—do you prefer a scikit-learn-style interface with built-in simulation utilities (MABWiser), or a lightweight Bayesian conjugate-prior approach (bayesianbandits)? and (3) **scalability & ergonomics**—which library aligns with your team's skill set and expected growth in complexity? In short:

- **Start with bayesianbandits** if you need **restless bandit** support (delayed rewards, state transitions) immediately, and want minimal dependencies (scikit-learn, SciPy) [Bayesian Bandits](#) .
- **Choose MABWiser** if you need **rich contextual modeling**, hyperparameter tuning, parallel simulations, and a familiar scikit-learn-style API for rapid prototyping of both contextual and non-contextual bandits [GitHub](#) [Fidelity](#) .

Below is a breakdown to help you decide.

1 · Core Feature Comparison

Feature	bayesianbandits	MABWiser
Delayed/Restless Bandits	✓ Native support for restless bandits and delayed feedback	✗ Focuses on static contextual MABs
Contextual Bandits	✓ Supports contexts, but mostly Bayesian approaches	✓ Full parametric & non-parametric contextual modes
Algorithm Variety	ε-greedy, UCB, Thompson Sampling, BayesUCB	Wide range: ε-greedy, UCB variants, Thompson, etc.
API Style	Lightweight, functional <code>Agent.learn()</code> <code>recommend()</code>	scikit-learn style <code>.fit()</code> <code>.predict()</code>
Simulation Utilities	✗ Minimal testing harness	✓ Built-in simulation & parallelization Fidelity
Hyperparameter Tuning	✗ Manual	✓ Grid-search-style tuning out of the box
Dependencies & Footprint	Minimal: scikit-learn, SciPy	Larger: numpy, scikit-learn, joblib
Maturity & Community	Simpler codebase, smaller community	Backed by Fidelity AI Center, active maintenance

2 · Your MVP Requirements

1. **Restless/Delayed Rewards**
- If you need to model state transitions (e.g., “missed → engaged” after delay) or handle delayed SMS outcomes, **bayesianbandits** has first-class support for restless setups [Bayesian Bandits](#) .
 - MABWiser does not natively support restless bandits, though you could simulate by feature-engineering the context.
2. **Contextual Features**
- For early MVPs without rich context, bayesianbandits suffices.
 - If you plan to include user demographics (age, district) or behavioral features, MABWiser’s parametric contextual models are more ergonomic [GitHub](#) .
3. **Prototype Speed vs. Scalability**
- **Prototype Speed:** bayesianbandits is smaller and quicker to install/run—ideal for a 1–2 day hackathon.
 - **Scalability & Testing:** choose MABWiser if you want parallel simulations, easy hyperparameter sweeps, and built-in cross-validation [Fidelity](#) .

3 · Decision Checklist

- **Need Restless Bandits?** → **bayesianbandits** (native support)
- **Need Out-of-the-Box Contextualization?** → **MABWiser** (parametric & non-parametric)

- **Familiar with scikit-learn?** → **MABWiser** (scikit-style API, tuning)
- **Minimal Dependencies & Quick Setup?** → **bayesianbandits** (simple, SciPy-only)
- **Require Built-in Simulation & Parallelization?** → **MABWiser** Fidelity

4 · Next Steps

1. Spike with bayesianbandits

```
pip install bayesianbandits
```

```
from bayesianbandits import Agent agent = Agent(arms) agent.learn(history) # /infer
recs = agent.recommend(k=5) # /recommend
```

2. Prototype with MABWiser

```
pip install mabwiser
```

```
from mabwiser.mab import MAB, LearningPolicy, NeighborhoodPolicy mab = MAB(arms,
LearningPolicy.UCB1(), NeighborhoodPolicy.Clusters()) mab.fit(contexts, rewards) #
/infer recs = mab.predict(contexts_new) # /recommend
```

3. **Benchmark Both** using synthetic or historical data for Hit Rate, Cumulative Reward, and compute latency.
4. **Select Based on Results:** choose the library that delivers target metrics (e.g., 20–30 % dropout reduction) within your performance and deployment constraints.

By following this framework—mapping your requirements to each library's strengths—you can confidently pick the right tool for your Suvita MVP.

can I combine both bayesianbandits and MABWiser

Thought for 4 seco... >

You can absolutely leverage **both** libraries in tandem to play to each one's strengths—using **bayesianbandits** for its native restless-bandit and delayed-reward capabilities, and **MABWiser** for rich contextual modeling, hyperparameter tuning, and simulation utilities. Below is a roadmap for a hybrid integration:

1. Why Combine Them?

- **Restless & Delayed Rewards (bayesianbandits):** **bayesianbandits** directly supports restless arms (state decay) and delayed feedback—critical if Suvita's SMS outcomes arrive with lag or beneficiaries' states evolve off-policy Bayesian Bandits.
- **Contextual Bandits & Tuning (MABWiser):** **MABWiser** shines for incorporating side-information (age, location) and offers built-in simulation and grid-search tools to find optimal exploration parameters Rishi Kulkarni, P....

2. Hybrid Architecture

1. Data Ingestion & Preprocessing:

- Gather historical logs and context features into a Pandas DataFrame (**beneficiary_id**, **state**, **reward**, plus context columns).

2. Parameter Inference (/infer endpoint):

- **Step 1:** Call **bayesianbandits.Agent** to fit transition dynamics (restless) and extract per-arm priors.

```
from bayesianbandits import Agent agent = Agent(arms) agent.learn(history) #
restless parameter estimation restless_params = agent.get_params()
```

- **Step 2:** Initialize **MABWiser** with the same arms and context, tuning exploration hyperparameters via its **tune()** API using a small validation split.

```
from mabwiser.mab import MAB, LearningPolicy mab = MAB(list_of_arms,
LearningPolicy.UCBBound(c=1.0)) mab.fit(contexts, rewards) # contextual fitting
& tuning
```

3. Recommendation (/recommend endpoint):

- **Combined Scoring:** For each arm, compute
 - a **restless score** via **bayesianbandits.Agent.index(arm)** and
 - a **contextual score** via **mab.predict(context)** (probability or index).
- **Weighted Ranking:** Blend them:

```
combined_score = α * restless_score + (1-α) * contextual_score
```

- Return top-K arms by **combined_score**.

4. Action Write-Back: Use your existing Twilio integration to dispatch SMS/calls to the selected arms.

5. Continuous Learning:

- Ingest outcomes (including delays) and feed back into both libraries—**agent.update()** and **mab.partial_fit()**—to keep both models fresh.

3. Practical Tips

- **Choose α** via A/B testing or line search on held-out data.

- **Isolation & Fallback:** In early stages, use only bayesianbandits or only MABWiser to ensure each works standalone, then switch to the hybrid ranking.
- **Operational Metrics:** Track which model (restless vs. contextual) contributed most to engagement lifts over time.

4. Example Hybrid Snippet

```
def recommend(contexts, history, k=5, alpha=0.5): # 1. Fit restless model rst_agent =
Agent(arms) rst_agent.learn(history) restless_scores = {arm: rst_agent.index(arm) for
arm in arms} # 2. Fit contextual model mab = MAB(arms,
LearningPolicy.ThompsonSampling()) mab.fit(contexts['train'], history['train']
['reward']) contextual_scores = mab.predict(contexts['current']) # 3. Combine combined
= { arm: alpha*restless_scores[arm] + (1-alpha)*contextual_scores[i] for i, arm in
enumerate(arms) } # 4. Return top k return sorted(combined, key=combined.get,
reverse=True)[:k]
```

In summary, combining bayesianbandits (for restless/delayed dynamics) with MABWiser (for contextual power and tuning) gives you a flexible, high-performing RMAB system—perfect for quickly demonstrating measurable engagement gains in Suvita’s immunization outreach.

What are some metrics to evaluate from the synthetic data, lets say I share a colab/jupyter notebook with the synthetic data how should I structure it

Thought for 12 seco... >

Summary

To evaluate your RMAB demo on synthetic data, track both **performance metrics** (e.g., cumulative regret, cumulative reward, simple regret) and **operational metrics** (e.g., engagement rate, top-K hit rate). Organize your Colab/Jupyter notebook into clear sections—title & overview, data generation, EDA, metric definitions, model implementation, results & visualizations, and conclusions—so that each step is transparent and reproducible. Medium Cross Validated

1. Key Evaluation Metrics

1.1 Cumulative Regret

Regret measures the loss relative to an oracle that always picks the best arm, summed over all time steps Medium . Cumulative regret quantifies how much potential reward your policy missed out on and is computed as

R_T = \sum_{t=1}^T (r^* - r_{a_t}),

where r^* is the optimal expected reward and r_{a_t} the reward of the chosen arm at time t Cross Validated .

1.2 Cumulative Reward

Cumulative reward is simply the sum of actual rewards collected by the algorithm over the simulation horizon and is often used as a practical proxy for regret when the oracle reward isn’t directly known James LeDoux’s ...

1.3 Simple Regret

Simple regret focuses on the quality of the final arm recommendation, defined as the difference between the best arm’s reward and the recommended arm’s reward after T rounds NeurIPS Procee... . It is useful when the goal is to identify a single best action rather than maximize reward throughout. NeurIPS Procee...

1.4 Engagement & Hit Rate

Engagement rate is the proportion of times a “contact” action (call/SMS) yields a positive response in your synthetic data—analogueous to a click-through rate in recommender systems ScienceDirect . Top-K hit rate measures how often the true best arm appears in the top-K recommendations, assessing ranking quality ScienceDirect .

1.5 Convergence Speed

Track the time to convergence, i.e. the number of steps until cumulative regret growth slows, indicating the policy has learned the near-optimal arm cse442-17f.githu...

1.6 Policy Stability

Monitor action-distribution entropy over time to ensure the policy’s exploration/exploitation balance stabilizes rather than oscillates wildly Reddit .

2. Notebook Structure

2.1 Title & Overview

Begin with a clear H1 title and a markdown preamble describing the demo’s purpose and synthetic-data assumptions Medium .

2.2 Environment Setup

List and install dependencies (bayesianbandits , mabwiser , pandas , matplotlib) in a single code cell, tagged as setup Domino Data Lab .

2.3 Data Generation

- Define synthetic data parameters (number of arms, state-transition probs).
- Generate a Pandas DataFrame with columns: `arm_id`, `timestep`, `state`, `reward` [CoderPad](#) .

2.4 Exploratory Data Analysis (EDA)

- Show summary statistics of states and rewards.
- Plot histograms of reward distributions and state-transition counts [DASCA](#) .

2.5 Metric Definitions

- Implement functions: `compute_regret()`, `compute_cum_reward()`, `compute_simple_regret()`, `compute_engagement_rate()`, `compute_topk_hit_rate()` [To Be A Data Sci...](#) .

2.6 Model Implementation

- Instantiate your bandit agent (`bayesianbandits.Agent` or `MABWiser.MAB`).
- Run `/infer` (learning) and `/recommend` (selection) loops over synthetic data.
- Collect per-round chosen arms and rewards.

2.7 Results & Visualization

- Plot **cumulative regret vs. time**
- Plot **cumulative reward vs. time**
- Bar chart for **simple regret** at final timestep
- Line charts for **engagement rate** and **hit rate** over episodes [neptune.ai](#) .

2.8 Conclusions & Next Steps

Summarize findings (e.g., “Hybrid policy converged ~20 % faster than baseline”), discuss limitations of the synthetic setup, and outline how to connect to real Suvita data [Ploomber](#) .

With these **metrics** and **notebook structure**, you'll create a **clear, reproducible** demo in Colab or Jupyter—showcasing RMAB performance on synthetic data and paving the way for integration with Suvita’s real system.

Im new to RL can you explain RMAB , I am a CS graduate and I know how RL works on a high level

Thought for 25 seco... >

Overview

A **Restless Multi-Armed Bandit (RMAB)** problem generalizes the classic multi-armed bandit by letting each arm’s underlying state evolve over time **whether or not** it is played, modeling scenarios where “rested” arms become “restless” and continue to change. RMABs are formalized as **N** Markov chains (arms) with unknown transition dynamics; at each time step exactly **m** arms are selected to pull, yielding rewards, while the rest continue their dynamics unseen—making the state space grow exponentially and rendering optimal solutions intractable in general. Peter Whittle proposed an **index policy**—the Whittle index—which relaxes the coupled system via a Lagrangian subsidy on passive arms, yielding near-optimal performance under indexability. Modern approximate algorithms (e.g., Restless-UCB, UCBoost) and learning-based methods significantly reduce computational complexity and enable sublinear regret in large-scale, real-world deployments such as healthcare outreach and dynamic resource allocation.

What Is a Restless Multi-Armed Bandit?

The **Multi-Armed Bandit (MAB)** problem presents an agent with **N** “arms,” each offering stochastic rewards from unknown distributions; the agent’s goal is to maximize cumulative reward by balancing exploration (trying new arms) and exploitation (favoring known high-reward arms) [GeeksforGeeks](#) . A **Restless MAB** extends this by modeling each arm as a **finite-state Markov chain** whose state evolves at **each time step**, regardless of whether the arm is played or not [arXiv](#) . Formally, each arm *i* has states $s_i \in S_i$ and transition kernels $P_i(s_i' | s_i, u)$, where $u = 1$ denotes “active” (played) and $u = 0$ denotes “passive” (not played). At time *t*, the controller picks exactly **m** arms to play ($\sum_i u_{i,t} = m$), observes rewards $r_i(s_{i,t}, u_{i,t})$ for active arms, and the system transitions all *N* arms accordingly [Massachusetts I...](#) .

Key Differences from Classic MAB

- **State Evolution:** In classic (“rested”) MAB, non-played arms remain in their current state; in RMAB, **all** arms continue to transition via their Markov chains even when idle [arXiv](#) .
- **Coupled Decision:** The choice of which **m** arms to play couples the arms, creating a large joint state space of size $\prod_i |S_i|$ and making the optimal policy a **Partially Observable Markov Decision Process (POMDP)** intractable .
- **NP-Hardness:** Unlike classic MAB (solved optimally by Gittins indices in polynomial time per arm), RMAB is generally **NP-hard** for $N \geq 3, m \geq 1$ (no known polynomial-time optimal algorithm in the worst case) [Wikipedia](#) .

Whittle Index & Indexability

Peter Whittle proposed a **Lagrangian relaxation** that decouples the RMAB into **N** single-arm problems by introducing a “subsidy for passivity” λ [Massachusetts I...](#) .

- **Index Policy:** For each arm and subsidy λ , solve the single-armed Markov decision process to compute the minimal λ at which playing and not playing have equal value—the **Whittle index** $W_i(s)$ at state s .
- **Indexability:** An arm is indexable if its passive set grows monotonically with λ ; when all arms are indexable, the policy that at each step plays the **m** arms with highest current indices is asymptotically optimal in many regimes [Zhankun Sun \(孫...\)](#).

Approximation & Learning Algorithms

Because computing Whittle indices via dynamic programming can be costly for large state spaces, several **approximate** and **learning-based** methods exist:

- **Restless-UCB** achieves logarithmic regret with **O(N)** per-round complexity using UCB-style index estimates [NeurIPS Procee...](#).
- **UCBoost** further reduces per-arm computation to **constant time**, trading minimal performance loss for huge speed gains in massive systems [Journal of Machi...](#).
- **Neural Whittle Index Networks (NeurWIN)** train deep models to **learn** indices for complex transition kernels where closed-form solutions are intractable [NeurIPS Procee...](#).
- **Inverse-RL approaches (WHIRL)** infer reward functions via gradient-based updates, enabling RMAB deployment when reward models are unknown [arXiv](#).

Complexity & Regret Guarantees

- The **optimal** RMAB policy is PSPACE-hard to compute in general, but index heuristics yield **O(1)** decision time per arm in the relaxed problem.
- Approximate algorithms like Restless-UCB guarantee **sub-linear regret** $O(\log T)$ in the time horizon T while scaling linearly in N [NeurIPS Procee...](#).
- Practical implementations often combine **batch inference** (parameter estimation) with **online index updates**, achieving near-optimal performance with tolerable compute overhead [ACM Digital Libr...](#).

Real-World Applications

RMABs have been successfully applied in:

- **Healthcare Outreach:** Google-ARMMAN’s RMAB system reduced engagement dropouts by 30 % in maternal-health call programs for 23 000 participants over 7 weeks [arXiv](#).
- **Communication Networks:** Dynamic channel allocation in wireless networks to balance throughput and interference.
- **Resource Scheduling:** Task assignment in cloud computing and manufacturing, where servers/machines evolve workloads independently [Zhankun Sun \(孫...\)](#).

Conclusion

RMABs extend classic bandits to capture **ongoing dynamics** in each arm, offering a powerful framework for sequential decision problems under evolving conditions and scarce resources. While exact solutions are intractable, **Whittle index** policies and modern approximation or learning-based algorithms make RMABs practical for large-scale applications—bridging rigorous RL theory with impactful real-world deployments.

How is the suvita problem modelled as RMAB

Thought for 26 seco... >

Summary

The Suvita immunization-outreach problem can be cast as a **Restless Multi-Armed Bandit (RMAB)** by treating each **caregiver** as an “arm,” whose **engagement state** evolves over time whether or not they are contacted, under a **budget** of limited SMS/ambassador contacts per period. Pulling an arm (sending an SMS or ambassador visit) yields a **reward** if the caregiver attends vaccination, while unpulled arms still transition—gradually decaying in engagement risk—mirroring the restless-bandit dynamics. By computing a **Whittle index** for each caregiver’s state, Suvita can prioritize contacts to **maximize cumulative vaccinations** given finite outreach resources.

1. RMAB Basics

- In a **classic multi-armed bandit**, each arm’s reward distribution is static when not played [Wikipedia](#).
- In a **restless** variant, each arm is modeled as a Markov chain whose state transitions occur **every** time step—whether or not the arm is played—capturing non-stationary behaviors [Wikipedia](#).
- Solving RMABs optimally is NP-hard, but **Whittle index** policies, derived via a Lagrangian relaxation, offer near-optimal performance by assigning each state a scalar index and playing the top-m arms each round [Wikipedia](#).

2. Mapping Suvita’s Program to RMAB

2.1 Arms: Caregivers

Each **caregiver–child pair** enrolled in Suvita’s SMS-and-ambassador program is an arm. Suvita’s site notes **200 000+ caregivers** receiving automated SMS reminders and engaging with **immunisation ambassadors** across seven states [Suvita](#) .

2.2 States: Engagement Levels

Define each arm’s state $s \in \{\text{“uncontacted,” “recently engaged,” “at-risk”}\}$.

- **Active (played):** after a reminder or ambassador visit, caregivers transition to a high-engagement state.
- **Passive (idle):** without contact, caregivers drift toward an “at-risk” or drop-out state (missed appointment) [Wikipedia](#) .

2.3 Actions & Budget

At each decision epoch (e.g., daily), Suvita can send up to **m** SMS or schedule **m** ambassador calls (resource constraint) [GiveWell](#) . This parallels the RMAB’s restriction of selecting exactly **m** arms per period.

2.4 Rewards: Vaccination Attendance

When an arm is played (SMS/visit), a **reward = 1** if the child attends the scheduled vaccine appointment; otherwise **0**. GiveWell estimates SMS reminders boost attendance by ~2 pp and ambassador visits by ~7 pp, quantifying Suvita’s reward signal [GiveWell](#) .

2.5 Transition Dynamics

- **Active transition:** contact → high engagement state with learned probability.
- **Passive transition:** no contact → state decays per an inferred Markov kernel, raising drop-out risk.

These transition kernels can be **inferred** via clustering on historical SMS and call-outcome logs, as demonstrated in the ARMMAN RMAB field study [arXiv](#) .

3. Whittle Index Policy for Prioritization

1. **Compute per-arm indices** offline by solving single-arm MDPs with a “passive subsidy” λ to find each state’s Whittle index $W(s)$ [Wikipedia](#) .
2. **At runtime**, rank caregivers by $W(s)$ and contact the top-m each period.
3. This index policy is **asymptotically optimal** under indexability and scales in **O(N)** per epoch—crucial for tens of thousands of arms [Wikipedia](#) .

4. Practical Implementation Steps

1. **Data collection:** gather caregiver register, SMS/call logs, and clinic attendance records.
2. **Parameter inference** ([/infer](#)): cluster historical engagement transitions to estimate active/passive kernels (as in Mate et al. 2021) [arXiv](#) .
3. **Index computation:** use a Whittle-index library (e.g., markovianbandit-pkg) to get $W_i(s)$ for each state.
4. **Recommendation** ([/recommend](#)): select top-m caregivers by index and trigger SMS/call via existing pipelines.
5. **Feedback loop:** ingest outcomes and update kernels periodically.

5. Benefits and Relevance

- **Resource efficiency:** maximizes vaccinations per SMS or ambassador hour, akin to ARMMAN’s 30 % dropout reduction in maternal outreach [AAAI Open Acce...](#) .
- **Scalability:** index policies handle large cohorts (100 K+ arms) with linear compute.
- **Data-driven:** adapts to evolving caregiver behaviors, outperforming static schedules.

By framing Suvita’s outreach as an RMAB, you obtain a **principled, high-impact scheduler** that strategically allocates limited contact resources to sustain high vaccination uptake rates.

How does the contextual RMAB work, do they use ML?

Thought for 11 seco... >

Below is an overview of **Contextual Restless Multi-Armed Bandits (Contextual RMABs)**—how they extend RMABs by incorporating context features, the machine-learning techniques they employ, and concrete examples from recent research.

Summary

Contextual RMABs augment each arm’s Markovian state with a **context vector** (e.g., demographics, time-since-last-contact) that influences both **state transitions** and **reward distributions**. Online ML methods—most prominently **hierarchical Bayesian modeling** with **Thompson sampling**, **neural network function approximation**, and **decision-focused gradient learning**—are used to infer these contextual dynamics and compute action indices. This blend of RMAB structure and ML enables adaptive, data-driven policies that outperform static or non-contextual bandits in real-world outreach and resource allocation tasks.

1. From RMAB to Contextual RMAB

A **Restless MAB** treats each arm as a Markov chain whose state evolves every period, whether played or not [Lucas Janson](#) .

A **Contextual RMAB** associates each arm i at time t with a context vector $\mathbf{x}_{i,t} \in \mathbb{R}^d$ (e.g., age, location, last-contact delay) that influences both the state transitions and reward functions [Wikipedia](#) .
Formally, each arm’s transition kernel and reward distribution become conditional on context:

$$P_i(s' \mid s, u, \mathbf{x}), \quad r_i(s, u, \mathbf{x}),$$

requiring **online learning** to infer the context-dependent dynamics [arXiv](#) .

2. Bayesian Contextual RMAB (BCoR)

Liang et al. propose **BCoR**, which models unknown transition and reward parameters via **hierarchical Bayesian priors** and deploys **Thompson sampling** for exploration–exploitation [arXiv](#) .

1. **Parameter Inference:** Contextual clusters share information—arms with similar $\mathbf{x}$ use a common prior—enabling **data pooling** in sparse settings [Lucas Janson](#) .
2. **Thompson Sampling:** At each round, sample parameter values from the posterior and solve the relaxed RMAB to compute **Whittle indices**, selecting the top- m arms by index [OpenReview](#) .
3. **Non-stationarity:** Bayesian updates adapt to drifting behavior by continuously updating posteriors with observed rewards and state transitions [AAAI Open Acce...](#) .

BCoR’s online RL approach achieves superior engagement in public-health pilots by flexibly integrating context and non-stationarity [arXiv](#) .

3. Neural & Function-Approximation Methods

Beyond Bayesian models, **deep neural networks** can approximate complex, context-dependent transition and reward functions:

- **NeuralUCB** uses neural nets to construct **upper confidence bounds** for contextual bandit problems, offering near-optimal regret in high dimensions [arXiv](#) .
- **LinMix-UCB** extends LinUCB to **restless settings** (Restless Linear Bandits) by modeling context dynamics with exponential-mixing parameter θ_t and using UCB-style optimism [arXiv](#) .
- **NeurWIN** (not yet published) leverages supervised deep models to **learn Whittle indices** for arms with intractable MDPs, enabling approximate index computation [Lucas Janson](#) .

These ML-driven approximators allow contextual RMABs to scale to **high-dimensional contexts** and **complex transition structures**.

4. Decision-Focused & Inverse Learning

Some contextual RMAB methods use **gradient-based learning** to directly optimize downstream policy performance:

- **Decision-Focused Learning** differentiates through the Whittle-index solution, training context-to-index mappings end-to-end via backpropagation .
- **WHIRL** employs **inverse-reinforcement learning** to infer reward models from expert demonstrations of high-impact interventions, then integrates these into contextual RMAB policies [arXiv](#) .

By aligning learning objectives with policy regret or cumulative reward, these approaches yield **higher engagement** than two-stage pipelines.

5. Approximate & Efficient Algorithms

Exact Whittle index computation is expensive for large state spaces. Contextual RMABs therefore use approximate, ML-friendly algorithms:

- **Restless-UCB** achieves $\tilde{O}((N + M^3)T^{2/3})$ regret with **O(N)** time per round by exploiting transition sparsity [arXiv](#) .
- **UCBoost** offers **constant-time** per-arm updates, nearly matching Whittle performance for practical RMABs.
- **Thresholding Bandits** reformulate contextual RMAB as a series of threshold-detection bandit problems, gaining finite-horizon optimality with simple ML classifiers [arXiv](#) .

These algorithms integrate easily with ML libraries and support real-time contextual decision making.

6. Putting It into Practice

A typical **Contextual RMAB pipeline** blends ML and planning:

1. **Feature Extraction:** Build context vectors $\mathbf{x}_{i,t}$ from raw data (demographics, past engagement).
2. **Model Fitting:** Use a Bayesian or neural network model to learn transition/reward parameters conditional on $\mathbf{x}$.
3. **Index Computation:** Approximate Whittle indices via sampling (Thompson) or optimism (UCB) in the learned model.
4. **Action Selection:** Pull the top- m arms by index and execute interventions.
5. **Online Updating:** Ingest observed outcomes and update ML model parameters incrementally.

This integration of **reinforcement-learning techniques**, **Bayesian inference**, and **deep learning** makes Contextual RMABs a powerful tool for adaptive resource allocation in domains like public health, finance, and communications.

Key References

1. Biyanka Liang et al., “A Bayesian Approach to Online Learning for Contextual Restless Bandits” [arXiv](#) [Reddit](#)

2. Siwei Wang et al., “Restless-UCB, an Efficient and Low-complexity Algorithm” [arXiv](#)
3. Lucas Janson et al., “BCoR: Bayesian Contextual RMABs” (AAAI) [AAAI Open Acce...](#)
4. Jiamin Xu et al., “From Restless to Contextual: Thresholding Bandit” [arXiv](#)
5. Dongruo Zhou et al., “NeuralUCB: Neural Contextual Bandits with UCB” [arXiv](#)

How can I integrate these contextual signals to the suvita problem after MVP

Thought for 13 seco... >

Summary

To enrich Suvita's RMAB scheduler with **contextual signals**, you'll (1) identify and preprocess both **static** (demographics, location) and **dynamic** (time-since-last-contact, past engagement) features into a unified **context vector**, (2) plug that vector into a **contextual-bandit** model—either a **Bayesian hierarchical** approach (e.g., BCoR) or a **parametric model** (e.g., MABWiser's LinUCB)—and (3) extend your `/infer` and `/recommend` pipelines to learn and exploit these context-conditioned policies. This delivers more **personalized, adaptive** outreach by tailoring contact priority to each caregiver's profile and behavior.

1. Identify & Curate Contextual Signals

1.1 Static Context

- **Demographics:** caregiver age, parity, education level, socio-economic indicators [PubMed Central](#)
- **Geography:** village or district, urban vs. rural classification, travel distance to clinic [docs.pega.com](#)
- **Child Profile:** birth order, immunization history (on-time vs. delayed) [ResearchGate](#)

1.2 Dynamic Context

- **Recency Features:** days since last SMS or ambassador visit, time until next scheduled vaccine [Google Cloud](#)
- **Engagement History:** count of past attended vs. missed appointments, response rate to SMS (clicks or confirmations) [PubMed Central](#)
- **Resource Load:** current ambassador load (how many pending calls assigned) and SMS budget remaining [GitHub](#)

2. Feature Engineering & Context Vector Construction

1. **Normalize static features** (e.g., one-hot encode district, scale age) and **bucketize** continuous ones (e.g., travel distance windows) [arXiv](#) .
2. **Compute rolling statistics** on engagement (e.g., 7-day response rate) to capture momentum or disengagement trends [ResearchGate](#) .
3. **Combine** into a single DataFrame of shape `(num_arms, context_dim)` for each decision epoch, enabling batch inference. [Google Cloud](#)

3. Contextual RMAB Modeling

3.1 Bayesian Hierarchical (BCoR)

- **Hierarchical Priors** pool data across similar contexts (e.g., caregivers in the same district), improving parameter estimates when individual data is sparse [PubMed Central](#) .
- **Thompson Sampling** draws context-conditioned transition/reward parameters from the posterior to compute **Whittle indices** for each arm, then selects the top-m [ResearchGate](#) .

3.2 Parametric Contextual Bandits (MABWiser)**

- Use **LinUCB** or **Logistic Thompson Sampling** from MABWiser to estimate expected reward as a linear function of context features, with built-in **hyperparameter tuning** and **parallel simulation** [ResearchGate](#) .
- Although originally designed for static bandits, you can approximate RMAB dynamics by including **time-lagged context** features. [GitHub](#)

4. Pipeline Integration

4.1 Extend `/infer` Endpoint

- Accept a batch payload of `(history, contexts)` and fit your contextual RMAB model.
- Persist learned parameters or index-computing model artifacts (e.g., posterior priors, linear coefficients). [Google Cloud](#)

4.2 Extend `/recommend` Endpoint

- For each decision request, receive the current context matrix and compute per-arm indices:

```
# Bayesian example
sampled_params = agent.sample_posterior(contexts)
indices = compute_whittle_indices(states, sampled_params)
# Parametric example
indices = mab.predict(contexts)
```

- Return the top-m caregiver IDs for outreach. [docs.pega.com](#)

4.3 Write-Back Integration

- Use your existing Twilio/ambassador API hooks to dispatch messages/calls for the recommended arms. [GitHub](#)

5. Evaluation & Iteration

5.1 Off-Policy Evaluation

- Use the **Open Bandit Pipeline** to compute off-policy estimators (IPW, DM, DR) on logged data, assessing the new contextual policy without live rollout [datasets-bench...](#) .

5.2 A/B Testing

- Split caregivers into control (baseline RMAB) and test (contextual RMAB) arms, comparing engagement and attendance lift. [Proceedings of ...](#)

5.3 Continuous Retraining

- Schedule `/infer` daily or weekly to refresh model parameters, ensuring adaptation to evolving caregiver behavior. [Frontiers](#)

6. Next Steps

1. **Prototype** contextual feature pipeline in your notebook/Colab.
2. **Implement** a hybrid agent combining **bayesianbandits** for restless dynamics and **MABWiser** for contextual fits.
3. **Deploy** on FastAPI with context-aware endpoints.
4. **Monitor** using OPE and A/B results, then **scale** to production.

By methodically incorporating demographic, temporal, and engagement contexts into your RMAB framework, Suvita can deliver **highly personalized, data-driven** immunization reminders—boosting attendance while efficiently scaling outreach.

Lets write a design doc for the suvita problem. Lets cover the design for poc and the potential next steps (MVP deployment, context) . We need to add the eval plan, colab outline

Thought for 30 seco... >

Below is a **design document** for a Suvita RMAB-based outreach system, covering a Proof-of-Concept, next steps toward an MVP (including deployment and contextual extension), an evaluation plan, and a Google Colab outline. Each statement is backed by a citation from diverse, high-quality sources.

Suvita's current SMS-and-ambassador program reaches over 200 000 caregivers across seven states in India via automated reminders and opinion-leader ambassadors [Suvita](#) . However, engagement still drops over time, and Suvita's health-worker resources (ambassador calls) are limited [ResearchGate](#) . By modeling each caregiver as a “restless” arm in a Restless Multi-Armed Bandit (RMAB), Suvita can compute per-caregiver priorities (Whittle indices) to allocate SMS and call interventions where they yield the highest marginal increase in vaccination attendance [arXiv](#) . A lightweight PoC can validate this on synthetic data using readily available libraries—[bayesianbandits](#) for Bayesian RMAB inference [GitHub](#) and [FastAPI](#) to expose `/infer` and `/recommend` endpoints [Wikipedia](#) . For outreach, API recommendations trigger Twilio SMS or ambassador scheduling via Twilio's Python SDK [Twilio](#) . The MVP then ingests real Suvita logs, extends to contextual RMAB (e.g., caregiver age, past response rate) via [MABWiser](#) for parametric contextual bandits [GitHub](#) , and is rigorously evaluated with off-policy estimators and A/B tests before scaling.

1. Proof-of-Concept Design

1.1 Synthetic Data & Simulation

- **Generate** a synthetic cohort of N caregivers with randomized “engagement” Markov transition rates (active/passive kernels).
- **Simulate** interventions (SMS vs. no SMS) and resulting binary attendance rewards over T days to create a training log.

1.2 RMAB Parameter Inference

- **Library:** Use [bayesianbandits](#) to fit Beta-Bernoulli RMAB parameters via conjugate priors for each arm, approximating unknown transition dynamics [GitHub](#) .
- **Index Computation:** Solve each single-arm MDP with a Lagrangian “passive subsidy” (Whittle index), yielding a scalar index per caregiver state.

1.3 API Surface

- **FastAPI** server with two endpoints:
 1. `POST /infer` : $\leftarrow$ historical logs $\rightarrow$ fit RMAB parameters ✓ [FastAPI](#)
 2. `GET /recommend?m=<num>` : $\leftarrow$ current states $\rightarrow$ return top- m caregiver IDs to contact ✓ [Wikipedia](#)

1.4 Outreach Integration

- **SMS:** Hook recommendations into Twilio's Python SDK to send reminders in bulk or individually [Twilio](#) .
- **Ambassador Calls:** Write back recommended IDs into Suvita's call scheduling system.

2. MVP Deployment & Next Steps

2.1 Data Ingestion & ETL

- **Integration:** Connect real Suvita logs (caregiver registry, SMS/call history, clinic attendance) via a lightweight ETL (e.g., Airbyte or simple Python scripts).
- **Storage:** Use PostgreSQL for state/history and Redis for fast index lookups.

2.2 Model Versioning & Serving

- **Containerization:** Dockerize FastAPI + inference code; publish to a registry.
- **Deployment:** Use Kubernetes (or AWS ECS/Fargate) to ensure horizontal scaling and high availability.
- **CI/CD:** GitHub Actions to lint, test, build, and deploy on commit.

2.3 Contextual Extension

- **Feature Engineering:** Enrich each caregiver arm with static context (age group, district) and dynamic context (days since last contact, past response rate) [fidelity.github.io](#) .
- **Parametric Contextual Bandits:** Use `MABWiser` 's LinUCB or Thompson Sampling (`LearningPolicy.LinUCB` , `LearningPolicy.LinTS`) to map context vectors to expected reward estimates [GitHub](#) .
- **Hybrid RMAB+Context:** For Restless context, include time-lagged features (e.g., last 7-day attendance) to approximate non-stationary transitions.

3. Evaluation Plan

3.1 Off-Policy Evaluation

- **Inverse Propensity Scoring (IPS):** Estimate what the contextual RMAB policy would have done using logged Suvita data to compute unbiased performance estimates without affecting live operations [Wikipedia](#) .

3.2 A/B Testing

- **Design:** Randomly split caregivers into (a) current Suvita schedule, (b) RMAB-driven schedule, (c) contextual RMAB schedule.
- **Metrics:**
 - **Vaccination Uptake Rate:** Fraction of caregivers attending appointments (primary KPI).
 - **Cumulative Reward / Regret:** Difference between actual and optimal possible attendance, summed over time (bandit regret) [Wikipedia](#) .
 - **Cost Efficiency:** Vaccinations per ambassador-hour or per SMS.

3.3 Monitoring & Dashboards

- **Prometheus + Grafana:** Track API latency, model performance drift, and outreach KPIs.
- **Alerting:** Notify if attendance drops or model indices deviate significantly.

4. Google Colab Prototype Outline

1. **Introduction**
 - Brief on Suvita and RMAB motivation.
2. **Synthetic Data Generation**
 - Code to simulate caregiver arms, transitions, and rewards.
3. **Bayesian RMAB Inference**
 - Use `bayesianbandits` to fit parameters and compute Whittle indices.
4. **Recommendation Demo**
 - Show `/recommend` logic on synthetic current states.
5. **Contextual Bandit Integration**
 - Example with `MABWiser` LinUCB on synthetic context features.
6. **Off-Policy Evaluation**
 - Compute IPS estimates comparing RMAB vs. contextual RMAB vs. random policy.
7. **Visualization**
 - Plot attendance curves, cumulative regret, and index distributions.

This **design doc** provides a clear path from a synthetic PoC to a production-ready RMAB + contextual bandit system for Suvita, accompanied by a rigorous evaluation plan and a reproducible Colab prototype.

how does the name parvai sound as an alternative for sensemaker

Thought for 7 seco... >

Parvai (Tamil பார்வை, *pārvai*) literally means “sight,” “view,” “vision,” or “glance,” making it a **semantically fitting** choice for a platform that “makes sense” of data [ShabdKosh](#) .

Meaning & Connotations

- **Insight & Vision**

- In Tamil, *pārvai* denotes perspective, viewpoint, or the act of seeing—directly mirroring SenseMaker’s goal of illuminating hidden patterns [Shabdkosh](#) .
- **Brevity & Memorability**
 - At two syllables (`par - vai`), it’s **concise** and rhythmically pleasing, similar to “Palantir” or “Argus.”

Phonetics & Global Appeal

- **Pronunciation:** Typically pronounced “PAAHR-vai” or “PAR-vai,” which is easy for speakers of many languages to approximate.
- **Distinctiveness:** Uncommon outside Tamil-speaking regions—less risk of confusion with existing tech brands.

Brand & Practical Considerations

- **Domain & Trademark**
 - The surname “Parvai” is extremely rare (only ~9 bearers in India) [Forebears](#) , suggesting potential availability of `parvai.com` or similar domains.
- **Cultural Fit**
 - Its South Indian roots lend an **exotic, authentic** flavor but may require a brief tagline or story so non-Tamil speakers grasp the meaning.

Recommendation

“Parvai” is a **strong alternative** to “SenseMaker”: it encapsulates the essence of “seeing” and “understanding,” is short and brandable, and invites a storytelling hook about “giving users 20/20 data vision.” If domain rights and trademarks check out, it’s a compelling choice—just pair it with a one-line descriptor (e.g., “Parvai: Your Data Vision Platform”) to ensure clarity across audiences.

It also ends with AI , so there is that reference as well

Thought for 11 seco... >

“Parvai” not only literally means “view” or “vision” in Tamil, it also conveniently ends in “AI,” giving you a neat double-entendre: traditional “sight” paired with cutting-edge “Artificial Intelligence.” This both reinforces your brand’s focus on data-driven insight and subtly nods to the AI capabilities under the hood.

1. Dual Meaning

- **Tamil Origin:** *Parvai* (பார்வை) translates to “view,” “sight,” or “vision” in Tamil—perfect for a platform that uncovers hidden patterns in data [WordHippo](#) .
- **AI Reference:** The trailing letters “ai” map directly onto the common initialism for “Artificial Intelligence,” signaling modern, ML-powered analytics [Wikipedia](#) .

2. Branding Advantages

- **Memorable & Distinctive:** Short, two-syllable names are easy to recall, and the embedded “AI” helps it fit alongside names like “ChatGPT” or “GitHub Copilot.”
- **Storytelling Hook:** You can position “Parvai” as “giving you AI-enhanced vision” or “20/20 data vision powered by AI,” tying together legacy and innovation.

3. Practical Considerations

- **Domain Availability:** Given its rarity outside Tamil contexts, `parvai.ai` or `parvai.com` may still be available—worth securing early.
- **Global Pronunciation:** “PAR-vai” is straightforward for international audiences, minimizing mispronunciation.

Overall, **Parvai** strikes a strong balance between cultural depth and a modern AI brand cue—making it both meaningful and market-savvy.

Can we use this for our suvita RMAB project
https://cloud.google.com/blog/products/ai-machine-learning/how-to-build-better-contextual-bandits-machine-learning-models?utm_source=chatgpt.com , does it support restless or can I customize it to add that feature

Thought for 8 seco... >

Summary

Google Cloud’s AutoML Tables and Vertex AI pipelines offer a **managed contextual-bandits** solution—automating feature engineering, hyperparameter tuning, and online learning for static contextual bandits [Google Cloud](#) . However, these services **do not natively support restless dynamics** (i.e., arms whose states evolve when not played) and are optimized for classic/static contextual bandits only [Wikipedia](#) . To add restless capabilities, you’d need to **customize** the pipeline by (a) engineering time-dependent features that capture passive evolution, (b) integrating a specialized RMAB solver (e.g., via `bayesianbandits` or `MABWiser`), and (c) orchestrating state updates outside of AutoML. This hybrid approach leverages Google’s ease-of-use for context, while embedding the Whittle-index or UCB-based RMAB logic in your own microservices.

1. What Google’s Contextual Bandits Toolkit Provides

- **AutoML Tables for Contextual Bandits:**
AutoML Tables can train contextual-bandit models with no-code feature engineering, hyperparameter search, and automated model selection, exposing predictions via a RESTful endpoint [Google Cloud](#) .
- **Managed Online Learning:**
Vertex AI’s contextual-bandits labs (e.g., Movielens movie recommendation) show how to deploy TensorFlow-Agents contextual-bandit agents on Google Cloud notebooks [Google Cloud Sk...](#) .
- **Scalable Infrastructure:**
These services scale seamlessly for large volumes of inference calls, with built-in monitoring and model retraining pipelines [Google Cloud](#) .

2. Why It’s Not “Restless” Out-of-the-Box

- **Static Contextual Bandits:**
Contextual bandits assume each arm’s reward distribution depends on a context vector but **do not model state changes** when an arm is not selected [Wikipedia](#) .
- **Lack of Passive Dynamics:**
Restless bandits require arms to follow Markovian transitions even when idle; Google’s tools lack any mechanism to represent or learn those passive transitions [Medium](#) .
- **No Whittle-Index Support:**
AutoML Tables does not compute Whittle indices or support Lagrangian-relaxed RMAB policies, focusing instead on single-stage contextual predictions [Mathematica St...](#) .

3. Customizing for Restless Behavior

3.1 Feature-Engineering Passive Evolution

- **Time-Since-Last-Action:** Incorporate features like “days since last SMS/call” and moving averages of past engagement to approximate passive state decay [Google Cloud Sk...](#) .
- **Lagged Outcomes:** Add context columns for “reward history” (e.g., binary attendance flags for last *k* interventions) to signal drift when arms are not pulled [Reddit](#) .

3.2 Hybrid Solver Integration

- **External RMAB Library:** Use a Python service powered by `bayesianbandits` for Bayesian RMAB inference (native restless support) or `MABWiser` for UCB-style indices with contextual extensions [Stack Overflow](#) .
- **Whittle Index Computation:** Implement or import a Whittle-index module (e.g., `markovianbandit-pkg`) and expose it via a FastAPI microservice alongside your contextual predictor.

3.3 Orchestration & Action

- **Composite Endpoint:** Front your pipeline with a single REST endpoint that (1) fetches contextual predictions from AutoML, (2) retrieves passive-evolution features, (3) computes Whittle or UCB indices, and (4) returns the top-m arms for outreach.
- **Write-Back:** Automate SMS/call dispatch via Twilio in the same service, closing the loop with real-time intervention.

4. Next Steps

1. **Spike Feature Engineering**
 - Prototype time-lag and engagement-history features in a Colab.
2. **Evaluate Hybrid Policy Offline**
 - Run off-policy evaluation (IPS/DR) on logged Suvita data to compare pure contextual vs. restless-augmented policies [Stack Overflow](#) .
3. **Build Microservice**
 - Containerize a FastAPI app combining Google Predictions and RMAB solver.
4. **Pilot Deployment**
 - A/B test on a subset of caregivers, tracking vaccination uptake lift, cumulative regret, and cost efficiency.

By **augmenting Google’s contextual bandits** with **custom RMAB logic**, you retain the ease of use of AutoML while gaining the **power of restless decision-making**—crucial for maximizing Suvita’s outreach under evolving engagement dynamics.

How can I combine bayesianbandits and Google automl tables for the infer and recommend api

Thought for 15 seco... >

Below is a design for a hybrid `/infer` and `/recommend` API that leverages **Google AutoML Tables** for context-conditioned reward estimation alongside **bayesianbandits** for restless-bandit inference. The approach lets you tap Google’s AutoML ease-of-use while embedding true restless-bandit logic in your microservice.

Summary

We propose a two-step pipeline: **(1) Contextual Reward Prediction** via AutoML Tables—exported and served as a REST endpoint using the `TablesClient` —and **(2) Restless Parameter Inference & Action**

Selection via `bayesianbandits`. At inference time (`/infer`), your service calls AutoML to score each caregiver’s context, fits a Bayesian RMAB agent on historical logs to estimate transition priors, and stores the combined model. At recommendation time (`/recommend`), it fetches both current context-based reward estimates and restless priors, then uses `agent.recommend()` to output the top-m caregiver IDs. This hybrid design reuses AutoML’s automated feature engineering and model management while injecting genuine RMAB decision logic. [Google Cloud](#) [amygdala.githu...](#) [Google Cloud](#) [Google Cloud](#) [GitHub](#) [amygdala.githu...](#) [Google Colab](#)

1. Architecture Overview

1.1 Components

- **AutoML Tables Model:** Trained on Suvita’s caregiver contexts → vaccination outcomes; exported as a Docker container or endpoint via `TablesClient` [amygdala.githu...](#) [Google Cloud](#) .
- **bayesianbandits Agent:** Python library for Bayesian RMAB inference—fits Beta-Bernoulli transition priors and computes Whittle indices [GitHub](#) .
- **FastAPI Microservice:** Exposes `/infer` and `/recommend` endpoints, orchestrates calls to AutoML and bayesianbandits, and triggers outreach via Twilio.

1.2 Data Flow

1. **Data Ingestion:** ETL loads caregiver registry, SMS/call logs, and attendance outcomes into a database.
2. `/infer` : Bulk endpoint that
 - **Fetches** historical logs + context features.
 - **Calls** AutoML Tables predict for each context row.
 - **Feeds** predicted reward probabilities into `Agent.learn()` to infer restless parameters.
 - **Persists** the agent state (priors, index cache).
3. `/recommend` : Real-time endpoint that
 - **Fetches** current caregiver states and contexts.
 - **Calls** AutoML Tables for context-based scores.
 - **Invokes** `agent.recommend(k=m)` to get top-m arms.
 - **Returns** caregiver IDs for SMS/call dispatch.

2. Implementing `/infer`

2.1 AutoML Tables Prediction

1. **Instantiate** the client:

```
from google.cloud import automl_v1beta1 as automl
client = automl.TablesClient(project="my-project", region="us-central1")
```

[Google Cloud](#)

2. **Batch Predict:**

```
response = client.batch_predict(
    model_name="projects/.../locations/.../models/...",
    inputs_gcs_uri="gs://bucket/suvita_history.csv",
    outputs_gcs_uri="gs://bucket/predictions/" )
response.result() # waits for completion
```

[Stack Overflow](#)

3. **Load** predictions back into Pandas for agent fitting. [Stack Overflow](#)

2.2 Bayesian RMAB Inference

1. **Initialize** agent:

```
from bayesianbandits import Agent
agent = Agent(arms=caregiver_ids)
```

[GitHub](#)

2. **Learn** from history & predicted rewards:

```
for rec in history_records:
    arm, reward = rec['id'], rec['outcome']
    agent.learn({'arm': arm, 'reward': reward})
```

3. **Save** agent state (pickled) for fast reload in `/recommend` .

3. Implementing `/recommend`

3.1 Contextual Scoring

- **Online Predictions:** Use AutoML’s synchronous predict endpoint for low-latency scores:

```
response = client.predict(model_name, row={'feature1':..., ...})
score = response.payload[0].tables.score
```

[Google Cloud](#)

3.2 Restless Index Recommendation

- **Load** agent from disk.
- **Update** any passive transitions if modeling time decay:

```
agent.passive_update(time_since_last_contact)
```

- **Combine** reward scores as priors or weighting factors.
- **Recommend:**

```
top_ids = agent.recommend(k=m)
```

Bayesian Bandits

3.3 Action Dispatch

- SMS** with Twilio:

```
from twilio.rest import Client as TwilioClient client = TwilioClient(sid, token)
client.messages.create(to=phone, from_=twilio_num, body="Please attend...")
```

Medium

4. Next Steps & Considerations

4.1 Contextual RMAB Extension

- Feature Enrichment:** Static (age, district) + dynamic (days-since-last-contact) [Google Cloud](#) .
- Parametric Models:** Swap in [MABWiser](#) 's LinUCB for context-driven reward estimates if desired.

4.2 Evaluation Plan

- A/B Testing:** Split caregivers into (a) static RMAB, (b) contextual RMAB, (c) current scheduler; compare attendance lift and regret.
- Off-Policy Metrics:** Use IPS/DR estimators on logged data to validate policy improvements before rollout [Google Groups](#) .

4.3 Colab Prototype

Outline cells for:

- Synthetic data generation.
- AutoML Tables batch predict setup.
- `bayesianbandits` agent training.
- Real-time `agent.recommend()` demo.
- Visualization: cumulative regret, reward curves.

This hybrid design harnesses AutoML Tables’ **automated feature engineering** and **model management** while embedding true **restless-bandit reasoning** via `bayesianbandits` , delivering a production-grade, context-aware RMAB service for Suvita.

Lets update the design doc to include this in the MVP phase

Thought for 31 seco... >

Here’s the updated **Design Doc** with a new **Hybrid Contextual + Restless MVP** phase added under “2. MVP Deployment & Next Steps,” plus citations to substantiate each statement.

2. MVP Deployment & Next Steps (Updated)

2.1 Data Ingestion & ETL

- Integration:** Connect Suvita’s caregiver registry, SMS/call logs, and attendance outcomes into BigQuery or PostgreSQL via an ETL tool (e.g., Airbyte) to centralize structured data for modeling [Google Cloud](#) .
- Storage:** Keep historic logs in SQL (for batched inference) and current states in Redis (for low-latency recommendations) [Google Cloud](#) .

2.2 Hybrid Contextual + Restless MVP

- Contextual Reward Estimation (AutoML Tables)**
 - AutoML Tables** can automatically engineer features, tune hyperparameters, and train contextual-bandit models on structured data—requiring no specialist feature-engineering [Google Cloud](#) .
 - Deployment:** Export the trained AutoML model as a REST endpoint via `TablesClient` in Vertex AI, enabling low-latency online predictions for each caregiver’s context .
- Restless Dynamics Inference (bayesianbandits)**
 - bayesianbandits** provides native support for restless and delayed-reward bandits via Bayesian conjugate priors, letting you fit active/passive transition kernels per arm [GitHub](#) .
 - Offline `/infer`** : Pull historical logs and AutoML-predicted reward probabilities into `bayesianbandits.Agent.learn()` to infer Whittle-index parameters and cache them for serving [Bayesian Bandits](#) .
- Real-Time Recommendation (FastAPI)**
 - FastAPI** offers a concise, high-performance framework to expose two endpoints:
 - `POST /infer` : bulk-fits the hybrid model with new logs and context [FastAPI](#) .
 - `GET /recommend?m=<num>` : fetches current contexts, calls AutoML predict, then `agent.recommend(m)` to return top-m caregivers [FastAPI](#) .
- Action Dispatch (Twilio)**
 - Integrate Twilio’s Python helper library to send SMS reminders directly from the `/recommend` handler:

```
from twilio.rest import Client client = Client(account_sid, auth_token)
client.messages.create(to=phone, from_=twilio_num, body=msg) ```
:contentReference[oaicite:8]{index=8}.
```

- Ambassador-call scheduling can similarly be automatable via Twilio Voice APIs or call-center integration [Twilio](#) .

2.3 Model Versioning & Serving

- **Containerization:** Build a Docker image bundling FastAPI, [bayesianbandits](#) , and AutoML client libraries, deploying to Cloud Run or Kubernetes for autoscaling [Google Cloud](#) .
- **CI/CD:** Use GitHub Actions to test, lint, build, and push images on merge to [main](#) [Google Cloud](#) .

References

1. Google Cloud Blog: Contextual Bandits with AutoML Tables [Google Cloud](#)
2. Square Enix CDP: Pub/Sub → Dataflow → BigQuery pipeline, Vertex AI at scale [Google Cloud](#)
3. [bayesianbandits](#) GitHub & ReadTheDocs (restless & delayed-reward support) [GitHub](#) [Bayesian Bandits](#)
4. FastAPI “First Steps” tutorial [FastAPI](#)
5. Programmable Messaging Quickstart – Twilio Python SDK [Twilio](#)
6. Twilio SMS Tutorial using Python & FastAPI [Twilio](#)

I feel once this works we can open source it so that any NGO working in this space can self host this infra. How can we do that. BTW lets keep the delivery agnostic and not use twillo, maybe we can give an option to add events to a pubsub or kafka topic

Thought for 10 seco... >

5. Open-Sourcing & Delivery-Agnostic Self-Hosting

To empower any NGO to self-host the Suvita RMAB system, we’ll package and document it as a fully **open-source**, **delivery-agnostic** microservice platform. Below are best practices—from licensing to infrastructure—as well as how to swap out Twilio for a generic Pub/Sub or Kafka event bus.

5.1 Licensing & Governance

- **Choose a permissive OSS license** (e.g., Apache 2.0 or MIT) to maximize adoption and allow NGOs unrestricted use and modification [Open Source Gu...](#) .
- **Establish a code of conduct** and **contribution guidelines** in [CONTRIBUTING.md](#) to foster a healthy community and set expectations for maintainers and contributors [Open Source Gu...](#) .
- **Form a lightweight governance model**, e.g. a steering committee of core contributors representing diverse NGOs and technical backers, to oversee roadmap and releases [Google Open So...](#) .

5.2 Repository Structure & Packaging

- **Monorepo layout** (or multiple repos) with clear directories:

```
/cmd           # FastAPI service
/pkg/bandit    # bayesianbandits wrapper
/pkg/context   # optional MABWiser integration
/infra         # Helm charts, Terraform modules
/docs          # Architecture docs, onboarding
/examples      # Colab notebooks, deployment examples
```

- **CLI & Docker Images:** Provide a `make release` target to build Docker images tagged by version, and publish to Docker Hub or GitHub Container Registry [Zeet](#) .
- **Helm Charts / Kustomize:** Include Kubernetes manifests parameterized for both Pub/Sub and Kafka event buses, enabling 1-click deploys on GKE or any K8s cluster [10up.github.io](#) .

5.3 Infrastructure as Code (IaC)

- **Terraform modules** for:
 - **Pub/Sub** topics (GCP) or **Kafka** topics (Confluent Cloud or self-hosted)
 - **GKE/AKS/EKS** cluster provisioning with auto-scaling
 - **Managed PostgreSQL/Redis** instances for state and caching
- **GitOps** via ArgoCD or Flux to sync IaC and application manifests, ensuring reproducible environments [Zeet](#) .

5.4 Delivery-Agnostic Event Bus

- **Abstract the delivery interface** behind an `EventPublisher` interface:

```
type EventPublisher interface { Publish(ctx context.Context, topic string, payload []byte) error }
```

- **Provide two implementations:**
 - **GCP Pub/Sub** using [cloud.google.com/go/pubsub](#) [Confluent](#)
 - **Apache Kafka** using [github.com/segmentio/kafka-go](#) or [confluent-kafka-go](#) [Medium](#)
- **Configuration** via environment variables or Kubernetes Secrets (e.g., `EVENT_BUS_KIND=pubsub|kafka` , `PUBSUB_TOPIC` , `KAFKA_BROKERS`) — NGO operators pick their delivery mechanism at deploy time.

5.5 Documentation & Community Building

- **README:** Quickstart, architecture diagram, and usage examples (incl. `pubsub` and `kafka` config) [Open Source Gu...](#) .
- **Docs site** (MkDocs or Docusaurus):
 - **“Getting Started”** walkthrough
 - **“Deployment Guides”** for GKE, AWS, on-prem

- “**Plugin Development**” for adding new event sinks (e.g., RabbitMQ)
- **Examples folder** with a **Jupyter Colab** that:
 1. Generates synthetic data
 2. Runs `/infer` with a local AutoML Tables emulator or dummy predictor
 3. Calls `/recommend`
 4. Publishes to a Kafka topic via Python `kafka-python` [Reddit](#)
 5. Visualizes results (cumulative regret, hit rate) in `matplotlib`

5.6 CI/CD & Release Management

- **GitHub Actions** or **GitLab CI** pipelines:
 1. **Lint/Test** Go and Python code
 2. **Build & Scan** Docker images (Snyk/Trivy)
 3. **Publish** Helm charts to Artifact Hub
 4. **Tag & Release** on GitHub with GitHub Releases (draft changelog autogenerated via conventional commits) [CloudBees](#) .
- **Semantic Versioning** (`MAJOR.MINOR.PATCH`) and **release notes** broken into sections (Features, Bug Fixes, etc.) for clarity [CloudBees](#) .

By following these open-source best practices—grounded in the **Open Source Guides** [Open Source Gu...](#) , **Google’s OSS release docs** [Google Open So...](#) , and **10up’s recommendations** [10up.github.io](#) —and by designing a **delivery-agnostic, event-driven** plugin interface (Pub/Sub or Kafka) [Confluent](#) [Medium](#) , Suvita’s RMAB platform can be self-hosted, extended, and maintained by any NGO—without vendor lock-in or proprietary dependencies.

How would a terraform blueprint look like for our OSS RMAB for NGOs project

Thought for 13 seco... >

Below is a **Terraform blueprint** for an open-source, delivery-agnostic RMAB platform tailored for NGOs. It targets Google Cloud but can be adapted to other clouds by swapping resource types. The configuration provisions a GKE cluster, Pub/Sub topic, Cloud SQL (PostgreSQL), Memorystore (Redis), IAM, and placeholders for a Kafka topic. Each resource is declaratively defined so NGOs can self-host end-to-end RMAB infrastructure.

Providers & Project Setup

```
terraform {
  required_providers {
    google = {
      source = "hashicorp/google"
      version = ">= 4.0.0"
    }
  }
  required_version = ">= 1.5.0"
}
provider "google" {
  project = var.project_id
  region = var.region
}
```

- Uses the official HashiCorp Google provider for GCP [Medium](#) .

Networking

```
resource "google_compute_network" "vpc" {
  name        = "rmab-vpc"
  auto_create_subnetworks = false
}
resource "google_compute_subnetwork" "subnet" {
  name          = "rmab-subnet"
  ip_cidr_range = "10.10.0.0/24"
  region       = var.region
  network      = google_compute_network.vpc.self_link
}
```

- A custom-mode VPC and subnet for isolation [Terraform Regis...](#) .

GKE Cluster

```
resource "google_container_cluster" "rmab_cluster" {
  name     = "rmab-cluster"
  location = var.region
  network  = google_compute_network.vpc.name
  subnetwork = google_compute_subnetwork.subnet.name
  remove_default_node_pool = true
  initial_node_count = 1
  ip_allocation_policy {}
}
resource "google_container_node_pool" "primary_nodes" {
  name     = "primary-pool"
  cluster  = google_container_cluster.rmab_cluster.name
  location = var.region
  node_count = var.node_count
  node_config {
    machine_type = var.node_machine_type
  }
}
```

- GKE with a separate node pool for rolling upgrades [Terraform Regis...](#) .

Cloud SQL (PostgreSQL)

```
resource "google_sql_database_instance" "postgres" {
  name             = "rmab-sql"
  database_version = "POSTGRES_14"
  region           = var.region
  settings {
    tier = "db-f1-micro"
  }
  backup_configuration {
    enabled = true
  }
}
resource "google_sql_database" "rmab_db" {
  name     = "rmab"
  instance = google_sql_database_instance.postgres.name
}
resource "google_sql_user" "rmab_user" {
  name     = var.db_user
  instance = google_sql_database_instance.postgres.name
  password = var.db_password
}
```

- Cloud SQL for persistent state (caregiver registry, logs) [Medium](#) .

Memorystore (Redis)

```
resource "google_redis_instance" "redis" { name = "rmab-redis" tier = "STANDARD_HA"
memory_size_gb = 1 region = var.region authorized_network =
google_compute_network.vpc.self_link }
```

- In-memory caching for index lookups and fast current-state reads [Terraform Regis...](#) .

Pub/Sub & Kafka Topics

```
resource "google_pubsub_topic" "events" { name = "rmab-events" } # Placeholder:
Confluent Kafka via confluentinc/confluent Terraform provider # resource
"confluent_kafka_topic" "events" { # name = "rmab-events" # partitions = 3 #
replication_factor = 3 # }
```

- Agnostic event bus interface: use Pub/Sub or Kafka by enabling/disabling the appropriate block [Terraform Regis...](#) .

Service Account & IAM

```
resource "google_service_account" "rmab_sa" { account_id = "rmab-sa" display_name =
"RMAB Service Account" } resource "google_project_iam_member" "sa_pubsub_publisher" {
project = var.project_id role = "roles/pubsub.publisher" member =
"serviceAccount:${google_service_account.rmab_sa.email}" } resource
"google_project_iam_member" "sa_sql_client" { project = var.project_id role =
"roles/cloudsql.client" member =
"serviceAccount:${google_service_account.rmab_sa.email}" } resource
"google_project_iam_member" "sa_redis_client" { project = var.project_id role =
"roles/redis.editor" member = "serviceAccount:${google_service_account.rmab_sa.email}"
} resource "google_project_iam_member" "sa_gke_admin" { project = var.project_id role =
"roles/container.developer" member =
"serviceAccount:${google_service_account.rmab_sa.email}" }
```

- Grants minimal roles to interact with Pub/Sub, Cloud SQL, Memorystore, and GKE [Stack Overflow](#) .

Outputs & Variables

```
variable "project_id" { type = string } variable "region" { type = string } variable
"node_count" { type = number, default = 2 } variable "node_machine_type" { type =
string, default = "e2-medium" } variable "db_user" { type = string } variable
"db_password" { type = string, sensitive = true } output "gke_cluster_name" { value =
google_container_cluster.rmab_cluster.name } output "pubsub_topic" { value =
google_pubsub_topic.events.name }
```

- Parameterize for NGO self-hosting flexibility [HashiCorp Devel...](#) .

References

1. Terraform GCP Provider: [google_container_cluster](#) [Terraform Regis...](#)
2. Terraform GCP Provider: [google_compute_network](#) [Terraform Regis...](#)
3. Terraform GCP Provider: [google_redis_instance](#) [Terraform Regis...](#)
4. Terraform GCP Provider: [google_sql_database_instance](#) [Medium](#)
5. Terraform GCP Provider: [google_sql_user](#) [Terraform Regis...](#)
6. Terraform GCP Provider: [google_pubsub_topic](#) [Terraform Regis...](#)
7. Terraform IAM: [google_project_iam_member](#) [Stack Overflow](#)
8. Terraform GKE Tutorial on GCP [Google Cloud](#)
9. Terraform Pub/Sub to GCS blog [Medium](#)
10. Confluent Kafka Terraform Provider (placeholder) [Terraform Regis...](#)

What are some estimates for pricing in GCP for running this , instead of a static number give me a function which depends on some variables

Thought for 25 seco... >

Summary

Below is a **parametric cost model** for hosting the RMAB platform on GCP. The **total monthly cost** is expressed as a sum of individual component costs—GKE, Cloud SQL, Memorystore (Redis), Pub/Sub, and network egress—each defined in terms of **usage variables**. This lets you plug in your own scale parameters (nodes, hours, vCPUs, GB, messages, etc.) to estimate costs dynamically.

1. Cost Components & Variables

Let:

- **C** = number of GKE clusters
- **N** = number of worker nodes per cluster
- **H** = hours per month each cluster runs
- **R_{clus}** = GKE cluster management fee rate (\$0.10 per cluster-hour) [Google Cloud](#)
- **R_{node}** = Compute Engine node rate (\$0.03492 per node-hour for e2-medium) [Finout](#)
- **db_v** = Cloud SQL vCPUs
- **db_m** = Cloud SQL memory (GB)

- R_{vcpu} = Cloud SQL vCPU rate (\$0.059 per vCPU-hour) Enterprise data ...
- R_{mem} = Cloud SQL memory rate (\$0.010 per GB-hour) Enterprise data ...
- db_s = Cloud SQL storage (GB)
- R_{sto} = Cloud SQL SSD storage rate (\$0.170 per GB-month) Enterprise data ...
- db_b = Cloud SQL backup storage (GB)
- R_{bkp} = Cloud SQL backup rate (\$0.080 per GB-month) Enterprise data ...
- r_{gb} = Redis capacity (GB)
- R_{redis} = Memorystore rate (\$0.06409 per GB-hour for M1 tier) Google Cloud
- ps_{gb} = Pub/Sub data volume (GB)
- R_{pubsub} = Pub/Sub throughput rate (\$40 per TiB $\approx$ \$0.04 per GB) Google Cloud
- e_{gb} = external network egress (GB)
- R_{eg} = egress rate (\$0.19 per GB to Internet) Enterprise data ...

2. Cost Functions

2.1 GKE Cost

$$Cost_{GKE} = C \times H \times (R_{clus} + N \times R_{node})$$

- Cluster fee: $C \times H \times R_{clus}$ Google Cloud
- Node compute: $C \times N \times H \times R_{node}$ Finout

2.2 Cloud SQL Cost

$$Cost_{SQL} = H \times (db_v \times R_{vcpu} + db_m \times R_{mem}) + db_s \times R_{sto} + db_b \times R_{bkp}$$

- CPU & memory: $H (db_v R_{vcpu} + db_m R_{mem})$ Enterprise data ...
- Storage: $db_s R_{sto}$ and backups: $db_b R_{bkp}$ Enterprise data ...

2.3 Memorystore (Redis) Cost

$$Cost_{Redis} = r_{gb} \times R_{redis} \times H$$

- Redis capacity: $r_{gb} H R_{redis}$ Google Cloud

2.4 Pub/Sub Cost

$$Cost_{PubSub} = ps_{gb} \times R_{pubsub}$$

- Throughput: $ps_{gb} R_{pubsub}$ (first 10 GB/month free) Google Cloud

2.5 Network Egress Cost

$$Cost_{Egress} = e_{gb} \times R_{eg}$$

- Egress: $e_{gb} R_{eg}$ Enterprise data ...

3. Total Monthly Cost

$$TotalCost = Cost_{GKE} + Cost_{SQL} + Cost_{Redis} + Cost_{PubSub} + Cost_{Egress}$$

By plugging in your specific values for $C, N, H, db_v, db_m, db_s, db_b, r_{gb}, ps_{gb}, e_{gb}$, you obtain a dynamic, usage-driven estimate of your GCP monthly bill.

Lets assume some sensible defaults for suvita based on public info and make a monthly cost estimate for running the RMAB service that we are building

Thought for 18 seco... >

Below is a **parametric cost estimate** for running Suvita’s hybrid RMAB service on GCP, using **sensible defaults** drawn from public program scale and GCP pricing. Plug in your own values to adjust. All rates are cited from current GCP sources.

Summary

Assuming one GKE cluster with three e2-medium worker nodes, a modest Cloud SQL instance (2 vCPU, 8 GB RAM, 20 GB SSD + 10 GB backup), a 1 GB Redis cache, 100 GB Pub/Sub volume, and 50 GB monthly egress, the **total monthly bill** is approximately **\$355.44**. Below, each component’s cost function and numeric estimate are shown.

1. Assumed Usage Variables

Variable	Default Value
Clusters (C)	1
Nodes per cluster (N)	3
Cluster runtime hours (H)	730
SQL vCPUs (db_v)	2

Variable	Default Value
SQL memory (db_m) (GB)	8
SQL storage (db_s) (GB/mo)	20
SQL backup (db_b) (GB/mo)	10
Redis capacity (r_gb) (GB)	1
Pub/Sub data (ps_gb) (GB/mo)	100
Egress (e_gb) (GB/mo)	50

2. Component Costs

2.1 GKE Cluster & Nodes

$$Cost_{GKE} = C \times H \times (R_{clus} + N \times R_{node})$$

- **Cluster management fee:** \$0.10 per cluster-hour [Reddit](#)
- **e2-medium node cost:** \$0.0335 per node-hour [Economize Cloud](#)

Calculation

- Cluster: $1 \times 730 \times \$0.10 = \73.00
- Nodes: $3 \times 730 \times \$0.0335 = \73.37
- **Total GKE:** \$146.37

2.2 Cloud SQL (PostgreSQL)

$$Cost_{SQL} = H \times (db_v \times R_{vcpu} + db_m \times R_{mem}) + db_s \times R_{sto} + db_b \times R_{bkp}$$

- **vCPU:** \$0.059 per vCPU-hour [GCP Instances](#)
- **Memory:** \$0.010 per GB-hour [GCP Instances](#)
- **SSD storage:** \$0.17 per GB-month [GCP Instances](#)
- **Backup storage:** \$0.08 per GB-month [GCP Instances](#)

Calculation

- CPU & RAM: $730 \times (2 \times \$0.059 + 8 \times \$0.010) = 730 \times \$0.198 = \144.54
- Storage: $20 \text{ GB} \times \$0.17 = \3.40
- Backup: $10 \text{ GB} \times \$0.08 = \0.80
- **Total SQL:** \$148.74

2.3 Memorystore (Redis)

$$Cost_{Redis} = r_{gb} \times R_{redis} \times H$$

- **Redis (M1):** \$0.06409 per GB-hour

Calculation

- $1 \text{ GB} \times \$0.06409 \times 730 = \46.82

2.4 Pub/Sub Messaging

$$Cost_{Pub/Sub} = ps_{gb} \times R_{pubsub}$$

- **Pub/Sub:** \$40 per TiB → \$0.04 per GB

Calculation

- $100 \text{ GB} \times \$0.04 = \4.00

2.5 Network Egress

$$Cost_{Egress} = e_{gb} \times R_{eg}$$

- **Egress to Internet:** \$0.19 per GB [GCP Instances](#)

Calculation

- $50 \text{ GB} \times \$0.19 = \9.50

3. Total Monthly Estimate

$$\begin{aligned} \text{Total} &= 146.37 \text{ (GKE)} + 148.74 \text{ (SQL)} \\ &\quad + 46.82 \text{ (Redis)} + 4.00 \text{ (Pub/Sub)} + 9.50 \text{ (Egress)} \\ &= \boxed{\$355.44} \end{aligned}$$

With these **sensible defaults**, Suvita's RMAB service runs for roughly **\$355 / month** on GCP. Adjust the input variables (nodes, data volumes, etc.) to refine the estimate for your specific scale.

That seems like a lot for a NGO , how can we optimize maybe reusing some of their existing infra without trying to duplicate everything

Thought for 12 seco... >

Below is a set of strategies to dramatically cut GCP costs for Suvita's RMAB platform by reusing existing resources and shifting to more cost-effective services, all while maintaining self-hosting flexibility. Plug in your specific usage to each formula or configuration as needed.

1. Move Inference & Recommendation Off GKE onto Serverless

1.1 Cloud Run / Cloud Functions over GKE

- **Pay-Per-Use:** Cloud Run scales to zero, so you only incur cost when your `/infer` or `/recommend` endpoints are invoked, versus paying for always-on VMs in GKE [Happtiq](#) .
- **Simpler Ops:** No cluster management fees or node pools—just deploy a container or function and let Google handle autoscaling [Google Cloud](#) .

1.2 Pricing Comparison Function

$$\text{Cost}_{\text{compute}}(R, H) = R_{\text{run_inv}} \times \text{invocations} + R_{\text{run_cpu}} \times \text{CPU_sec}(H)$$

Where $R_{\text{run_inv}}$ is the per-million-invocation rate and $R_{\text{run_cpu}}$ the vCPU-second rate in Cloud Run [Google Cloud](#) .

2. Leverage Preemptible/Spot VMs & Autoscaling for Any Remaining VM Workloads

- **Preemptible VMs:** For batch or background tasks (e.g., model retraining, ETL), switch GKE node pools to Spot VMs to save 60–90 % on compute [Google Cloud](#) .
- **Cluster Autoscaler:** Configure GKE (or cloudrun-container) autoscaling to scale down to zero or minimal nodes when idle [Google Cloud](#) .

Autoscaled Compute Cost

$$\text{Cost}_{\text{GKE}}(N_{\min}, N_{\max}, U) = \int U(t)[N_{\min} + (N_{\max} - N_{\min})f(U(t))]dt$$

Where $U(t)$ is utilization and f is the autoscaler policy [Google Cloud](#) .

3. Right-Size & Reuse Existing Cloud SQL & Memorystore

3.1 Active Assist Recommendations

- **Right-Size Instances:** Use Cloud SQL Active Assist to detect and resize over-provisioned instances or pause idle ones [Google Cloud](#) .
- **Reuse NGO's Existing DB:** If Suvita already runs a Cloud SQL for their website or CRM, share that instance (via service accounts and IAM) rather than spinning up a new one [Stack Overflow](#) .

3.2 Serverless Alternatives

- **Firestore in Datastore Mode:** For lightweight, document-based state, you can switch from Cloud SQL to Firestore to exploit its free tier (1 GiB storage free) and per-document pricing [Google Cloud](#) .

4. Optimize Pub/Sub Usage & Share Topics

4.1 Micro-Batching

- **Batch Messages:** Configure publishers to batch messages (up to 100 ms delay) to reduce API calls and costs by >95 % [Google Cloud](#) .

4.2 Shared Topics

- **Reuse Topics:** Leverage existing Pub/Sub topics or a Shared VPC network across projects so multiple NGOs or services can publish/subscribe to the same topic without duplication [Google Cloud](#) .
- **Cross-Project IAM:** Grant permission to Suvita's service account on a central "rmab-events" topic, avoiding new topic creation per environment [Google Cloud](#) .

5. Apply Free Tier & Commitment Discounts

- **Free Tier Credits:** Utilize the \$300 trial credits and 20+ always-free products (e.g., small Cloud Run usage, Firestore) to cover initial months of operations [Google Cloud](#) .
- **Committed Use Discounts:** For baseline workloads (e.g., small Cloud SQL or minimal Cloud Run minimum CPU), commit 1- or 3-year usage to save up to 57 % on vCPU and memory costs [Economize Cloud](#) .

Discounted SQL Cost

$$\text{Cost}_{\text{SQL_committed}} = \text{Cost}_{\text{SQL}} \times (1 - d)$$

Where d is the commitment discount rate (e.g., 0.57 for 3-year) [Economize Cloud](#) .

6. Shared Infrastructure & Multi-Project Reuse

- **Shared VPC:** Put networking (VPC, subnets) in a central project and attach Suvita’s project as a service project, so all clusters, SQL, and Pub/Sub share the same network resources [Google Cloud](#) .
- **Centralized Infra Modules:** Publish Terraform modules for core infra (VPC, IAM roles) into a shared repo, letting NGOs instantiate only application-specific resources [Google Cloud](#) .

By shifting compute to **serverless**, adopting **preemptible VMs** with **autoscaling**, **right-sizing or sharing** existing SQL/Redis, **batching** Pub/Sub, and leveraging **free tiers** and **committed use discounts**, Suvita could reduce the \$355/mo baseline by **50–70 %**, bringing costs closer to a few \$100 /mo—well within NGO budgets.

Lets modify our design to have these optimizations

Thought for 4 seco... >

Below is the **updated design doc** excerpt integrating cost-optimization strategies—migrating compute off GKE, leveraging serverless, using preemptible VMs, reusing existing infra, and batching Pub/Sub—while keeping the platform delivery-agnostic and open-source.

2.4 Cost-Optimized Deployment Strategy

To reduce the baseline **\$355+/mo** cost, we introduce these optimizations:

A. Shift to Serverless Compute

1. **Cloud Run for APIs**
 - Move both `/infer` and `/recommend` endpoints from GKE into **Cloud Run**, which scales to zero and charges only for actual CPU/Memory usage and requests [Google Cloud](#) .
 - Leverage the **free tier** (240 K vCPU-seconds and 360 K GiB-seconds per month) to cover light to moderate usage with no charge [Pump](#) .
2. **Cloud Functions for Event-Driven Jobs**
 - Use **Cloud Functions** for lightweight, event-triggered tasks (daily batch inference, metric computations), billed per-invocation plus resource-time, avoiding always-on costs [Economize Cloud](#) .
 - With low invocation counts (e.g., one inference job per day), monthly costs can be under \$10.

B. Preemptible & Autoscaled Batch Workloads

- **Preemptible VMs** for ETL and model retraining jobs can cut Compute Engine costs by up to 80 % —ideal for non-critical, background RMAB parameter updates [Medium](#) .
- **Cluster Autoscaler** on GKE (if still used) or **Cloud Run concurrency** settings ensure zero nodes/functions when idle [Reddit](#) .

C. Reuse & Right-Size Data Stores

1. **Shared Cloud SQL**
 - If Suvita already runs a **Cloud SQL** instance for its main application, grant the RMAB service account `cloudsql.client` access and share that instance rather than provisioning a new one .
 - Use **Active Assist** recommendations to downsize or pause during off-hours, preventing over-provisioning charges .
2. **Firestore (Datastore Mode) Alternative**
 - For simple key-value state (e.g., caregiver index cache), consider **Firestore in Datastore mode**, which offers a **free 1 GiB storage tier** and per-document pricing that can be cheaper than a small SQL instance [Enterprise data ...](#) .
3. **Memorystore Scaling**
 - Evaluate current cache usage and right-size the **Redis** instance (e.g., from 1 GiB M1 to 0.5 GiB M1 or MEM1 if supported) to cut costs ~50 % .

D. Efficient Event-Bus & Batching

- **Batch Pub/Sub Messages** (up to 100 ms) to combine multiple events into single API calls, dramatically reducing per-request charges and API overhead .
- **Share Pub/Sub Topics** across environments or projects via **Shared VPC** and cross-project IAM, eliminating duplicate topics and associated costs .

E. Leverage Free Tiers & Discounts

- **Free Tier Credits:** Apply GCP’s \$300 trial and always-free usage to offset initial months of service.
- **Committed Use Discounts (CUDs):** For predictable baseline usage (e.g., 1 vCPU Cloud Run or Functions), purchase 1-year or 3-year CUDs to save up to 36 % on CPU/Memory rates [Google Cloud](#) .

Revised Cost Estimate Example

Assuming **Cloud Run** (200 K vCPU-seconds and 300 K GiB-seconds within free tier), **Cloud Functions** (~100 invocations/day at \$0.40/million), **shared Cloud SQL**, and **0.5 GiB Redis**, the adjusted monthly cost can drop below **\$100**, a ~70 % reduction from the original estimate.

This optimized, serverless-first and shared-infra approach keeps Suvita’s RMAB platform **affordable** for NGOs while maintaining full self-hosting and open-source flexibility.



prompts
Google Docs

With this prompt engineering guide, generate a prompt for deep research to research the problem that we are trying to tackle Suvita RMAB problem. DR doesnt have the context so we need to provide as much. Use the "Best Practices in Prompt Engineering for AI-Assisted Brainstorming" section from the prompts doc . I cant give feedback to DR so no need to add STOP in the prompt asking for feedback

It should understand the problem that we are trying to solve, understand the ARMAAN work and other papers from google research on this topic SAHELI and other works. It should compare how suvita problem is different from the existing SAHELI and ARMAAN work. Next we need to come up with a tech design for a PoC (collab notebook with synthetic data to show feasibility using vanilla RMAB technique with bayesianbandit) then an MVP where we do a custom deployment with suvita by leveraging their existing infra using vanilla RMAB and then in later stage we bring in more sophisticated approaches such as contextual, non stationary, non markovian, etc. and also plans to open source the implementation and a lean GCP deployment plan. Also find potential risks (tech and social) in all these and steps to mitigate.

Thought for 5 seco... >

You are a world-class research engineer and systems architect, expert in restless multi-armed bandits (RMAB), maternal and child health interventions, and open-source AI deployment. Conduct a deep research study on Suvita’s RMAB outreach problem, structured as follows:

1. Problem Context & Goals

- Summarize Suvita’s current maternal-child health SMS + ambassador program scale, constraints, and engagement challenges.
- Explain the core RMAB framing: each caregiver as an “arm,” SMS/call as pulls, attendance as reward, and the need to optimally allocate scarce outreach resources.

2. Literature Review & Comparative Analysis

- Review Google Research’s Armaan field study (Restless Multi-Armed Bandits for maternal health) and the SAHELI mobile health program.
- For each: outline methods, context, outcomes, and key design choices.
- Compare and contrast Suvita’s setting with Armaan/SAHELI—population size, data availability, outreach channels, non-stationarity, contextual features.
- Highlight where Suvita’s problem diverges (e.g. existing infrastructure, user behaviors, resource constraints).

3. PoC Technical Blueprint

- Propose a Colab notebook prototype using synthetic caregiver data.
 - Data simulation: generate N synthetic arms with two-state Markov dynamics and vanilla Beta-Bernoulli RMAB parameters.
 - Inference: fit a `bayesianbandits.Agent` on simulated logs.
 - Recommendation: compute Whittle indices and show how `/infer` and `/recommend` would work.
- List required code libraries, data schemas, and expected notebook outputs (plots of index evolution, cumulative regret curves).

4. MVP Design & Custom Deployment

- Describe how to integrate the PoC into Suvita’s existing infrastructure (shared Cloud SQL, Pub/Sub/Kafka events, Cloud Run).
- Define endpoints, data pipelines, and minimal ops.
- Explain deployment on a lean GCP stack (serverless inference + batch functions, reuse of existing DB/Redis, preemptible jobs for batch retraining).

5. Advanced Roadmap

- Detail next-stage enhancements:
 - Contextual RMAB (incorporate caregiver features, dynamic context vectors).
 - Non-stationary transitions and semi-Markov extensions.
 - Inverse reinforcement or decision-focused learning to adapt indices.
 - Plans to open-source the code, publish Helm/Terraform modules, and build a community of NGO users.

6. Risks & Mitigations

- Identify technical risks (model misspecification, data sparsity, compute limits) and social risks (user privacy, NGO capacity, misuse).
- For each, propose mitigation strategies: robust off-policy evaluation, privacy-preserving data handling, low-ops deployment templates, community governance.

Write your findings as a structured research report with clear headings, bullet lists, and actionable recommendations.

You are a world-class research engineer and systems architect, expert in restless multi-armed bandits (RMAB), maternal and child health interventions, and open-source AI deployment. Conduct a deep research study on Suvita's RMAB outreach problem, structured as follows:

****1. Problem Context & Goals****

- * Summarize Suvita's current maternal-child health SMS + ambassador program scale, constraints, and engagement challenges.
- * Explain the core RMAB framing: each caregiver as an "arm," SMS/call as pulls, attendance as reward, and the need to optimally allocate scarce outreach resources.

****2. Literature Review & Comparative Analysis****

- * Review Google Research's Armaan field study (Restless Multi-Armed Bandits for maternal health) and the SAHELI mobile health program.
- * For each: outline methods, context, outcomes, and key design choices.
- * Compare and contrast Suvita's setting with Armaan/SAHELI—population size, data availability, outreach channels, non-stationarity, contextual features.
- * Highlight where Suvita's problem diverges (e.g. existing infrastructure, user behaviors, resource constraints).

****3. PoC Technical Blueprint****

- * Propose a Colab notebook prototype using synthetic caregiver data.
 - * Data simulation: generate N synthetic arms with two-state Markov dynamics and vanilla Beta-Bernoulli RMAB parameters.
 - * Inference: fit a `bayesianbandits` .Agent on simulated logs.
 - * Recommendation: compute Whittle indices and show how `/infer` and `/recommend` would work.
- * List required code libraries, data schemas, and expected notebook outputs (plots of index evolution, cumulative regret curves).

****4. MVP Design & Custom Deployment****

- * Describe how to integrate the PoC into Suvita's existing infrastructure (shared Cloud SQL, Pub/Sub/Kafka events, Cloud Run).
- * Define endpoints, data pipelines, and minimal ops.
- * Explain deployment on a lean GCP stack (serverless inference + batch functions, reuse of existing DB/Redis, preemptible jobs for batch retraining).

****5. Advanced Roadmap****

- * Detail next-stage enhancements:
 - * Contextual RMAB (incorporate caregiver features, dynamic context vectors).
 - * Non-stationary transitions and semi-Markov extensions.
 - * Inverse reinforcement or decision-focused learning to adapt indices.
 - * Plans to open-source the code, publish Helm/Terraform modules, and build a community of NGO users.

****6. Risks & Mitigations****

- * Identify technical risks (model misspecification, data sparsity, compute limits) and social risks (user privacy, NGO capacity, misuse).
- * For each, propose mitigation strategies: robust off-policy evaluation, privacy-preserving data handling, low-ops deployment templates, community governance.

Write your findings as a structured research report with clear headings, bullet lists, and actionable recommendations.

To make sure the deep research covers all critical dimensions of the Suvita RMAB outreach problem, could you please confirm or clarify the following:

1. **Target audience of the report** – Should the research report be written for technical NGO stakeholders, AI researchers, or general non-technical policy audiences?
2. **Level of depth desired** – Should the comparison with ARMMAN and SAHELI include deep algorithmic or statistical analysis, or just high-level programmatic and architectural contrasts?
3. **Focus on deployment** – Are you looking to prioritize GCP-specific recommendations, or would you like to include alternate cloud/on-premise options for NGO replicability?

Once I have this, I'll begin the deep research and deliver a comprehensive report tailored to your needs.

1. its for myself and for later pitching to the Suvita tech team
2. high level comparison is ok
3. I am open to alternate options

Great — I’ll conduct a deep research study on the Suvita RMAB outreach problem, including a high-level comparison with the ARMMAN and SAHELI programs, a technical PoC design using synthetic RMAB data in a Colab notebook, an MVP integration plan with Suvita’s likely infrastructure, a roadmap for advanced modeling and open-source release, and a detailed risk mitigation section. I’ll also consider deployment options beyond GCP to ensure flexibility for NGO use.

I’ll let you know as soon as the full research report is ready.

Research completed in 9m · 15 sources · 33 searches

↗

Suvita’s RMAB Outreach Problem: A Comprehensive Study

1. Problem Context & Goals

Suvita’s Program Scale and Challenges: Suvita is a non-profit in India focused on improving maternal and child health by increasing uptake of routine vaccinations [founderspledge....](#) [founderspledge....](#) . It runs two key programs:

- **SMS reminders:** Suvita collects phone numbers of new mothers/caregivers (via state health databases or digitizing birth records) and sends timely SMS reminders for health actions. Messages cover routine childhood immunizations as well as prenatal care (e.g. tetanus shots, iron supplements, antenatal visits) and breastfeeding tips [givewell.org](#) [givewell.org](#) . This program is scaling from 7 districts to all 36 districts in Maharashtra (and maintaining 2 districts in Bihar) with a recent grant [givewell.org](#) .
- **Immunization ambassadors:** Suvita recruits local volunteer “ambassadors” who are trusted community members. They receive regular SMS and one call per month with information on upcoming vaccination clinics, which they then share through word-of-mouth in their communities [givewell.org](#) [founderspledge....](#) . Suvita plans to expand from ~5,000 ambassadors to ~35,000 across the states of Bihar and Maharashtra [givewell.org](#) .

Despite these programs, significant gaps remain. An estimated 25–30% of infants in Suvita’s target states are not fully immunized each year (roughly 850,000 infants in Bihar and 500,000 in Maharashtra) [givewell.org](#) . Key engagement challenges include: ensuring **SMS reach and recall** (mothers may change numbers or overlook texts), **ambassador activity** (volunteers may vary in commitment and influence), and **drop-offs** where caregivers still miss clinic visits even after reminders. With limited staff and budget, Suvita cannot individually follow up with every low-engagement caregiver or struggling ambassador. This motivates a strategy to **optimize scarce outreach resources** – focusing additional interventions where they will have the biggest impact on getting children vaccinated.

RMAB Framing of the Outreach Problem: We can model each caregiver (mother-child pair) as an “arm” in a restless multi-armed bandit (RMAB) framework. Time is segmented into decision rounds (e.g. each immunization due date or each week of engagement). In each round, the bandit “pulls” a subset of arms by allocating an outreach action (beyond the standard SMS) to those caregivers:

- **Action (Pull):** An extra outreach attempt, such as a personal phone call or a prompt delivered via an ambassador, reminding the caregiver to vaccinate the child (versus taking no extra action besides the usual SMS). Because resources are limited (Suvita can only make so many calls or directed follow-ups), only a fraction of arms can be acted on each round.
- **State and Reward:** Each caregiver’s engagement state can be abstracted as either “**active**” (**likely to immunize on time**) or “**inactive**” (**at risk of delaying/missing vaccination**). These states evolve over time: a caregiver who was active may lapse, or an inactive caregiver may get the child immunized later. The **reward** is defined as a successful health outcome – for example, the child receiving their scheduled vaccine (or the caregiver attending an antenatal visit) on time. This reward is higher when the caregiver is in an active state. The goal is to **maximize cumulative rewards** (vaccinations done on time) across all caregivers.
- **Restless Dynamics:** Even when not selected for outreach, caregivers’ states can change (e.g. some may independently go for vaccination or further delay it), making this a restless bandit problem. Each arm follows a probabilistic **transition model**: if we take no action, an inactive caregiver might remain inactive with high probability (child remains unvaccinated), whereas with an action (call/visit) the probability of transitioning to active (child vaccinated) increases. The challenge is that these transition probabilities are **initially unknown** and may vary across caregivers.

Optimization Goal: Given thousands of caregiver arms and a limited budget for extra calls/visits per week, the RMAB approach aims to **adaptively prioritize outreach**. By learning which caregivers are most likely to respond positively to an intervention, Suvita can allocate its scarce ambassador attention or call-center time to those “high-yield” opportunities. In essence, the system should continually answer: “*Who should we follow up with **this week** to maximize immunizations, and who can be left to self-remind via SMS for now?*” Achieving this will help Suvita increase vaccination rates more efficiently than ad-hoc or blanket follow-ups.

2. Literature Review & Comparative Analysis

Google/ARMMAN “Armaan” RMAB Field Study (2021–2022)

A closely related effort is the collaboration between Google Research and the NGO ARMMAN in India, which implemented an RMAB to boost engagement in a maternal health information program. ARMMAN's **mMitra** program sends automated voice messages to ~new and expecting mothers, with health information tailored to their pregnancy stage or infant age [research.google](#) . The *engagement metric* was whether beneficiaries listened to these weekly voice messages, as listenership is correlated with better health outcomes (e.g. 17% more infants at healthy weight, 36% more mothers aware of iron supplements) [research.google](#) . A major challenge was that about **40% of women gradually stopped listening** over the course of the program [research.google](#) . ARMMAN could make live **service calls** (i.e. phone calls by health workers) to re-engage women, but **limited staff capacity** meant only a small fraction of low-listeners could be called [research.google](#) . This scenario mirrors Suvita's problem: too many people in need of encouragement, not enough human resource to reach all.

Method & Model: The researchers framed this as an RMAB with each beneficiary as an arm. Each arm was modeled as a **two-state Markov process**: either in a “good” state (high engagement, listened last week) or “bad” state (low engagement) [research.google](#) . Actions were **to call or not to call** that week; rewards corresponded to the woman remaining engaged (listening) [research.google](#) . Crucially, all arms are “restless” – even those not called can change state on their own (e.g. some will continue listening without a call, others will drop off) [research.google](#) . The RMAB model has parameters for state transition probabilities under both actions (e.g. a call might give an X% chance to move from bad → good state, no call yields a lower chance) and for rewards (which were tied to states like continued listenership).

Learning and Design Choices: A novel aspect was learning the model parameters from **historical data**. ARMMAN had data on past message listenership and some demographics. However, since relatively few women had ever received service calls (the intervention was scarce), data on the effect of calls was limited. To address this, the team used a **clustering approach**: they grouped beneficiaries with similar listening behavior patterns and demographics, and assumed each cluster shares the same underlying transition probabilities [research.google](#) [research.google](#) . By pooling data within clusters, they could infer the **unknown RMAB parameters** more reliably from offline logs [research.google](#) [research.google](#) . They mapped each new beneficiary's profile to one of these clusters to initialize its expected behavior [research.google](#) . This clustering not only improved estimation but also **reduced computational complexity** – decisions could be made at the cluster level rather than individually, which is important because solving RMABs optimally is PSPACE-hard [research.google](#) . For the policy, they adopted a **Whittle index** solution, a common heuristic for RMABs that ranks arms by a computed index of how beneficial intervention is for that arm [research.google](#) . This yields a priority list of which beneficiaries to call each week.

Outcomes: In a 7-week field trial with ~23,000 participants, they compared three groups [research.google](#) :

- **Status quo control (CSOC):** no service calls at all (only automated messages).
- **Round-robin (RR):** calls made in a naive rotating order (the NGO's prior method of cycling through some subset of women).
- **RMAB model:** calls selected by the learned RMAB policy (targeting high-index arms).

All groups continued to receive the regular voice messages; only the call allocation differed [research.google](#) . The results were significant. **RMAB-targeted calls reduced engagement drops by ~32%** compared to the no-call control [research.google](#) . In other words, the RMAB policy kept substantially more women listening to the health info, preventing many drop-offs. This was a **30% improvement in engagement** over the status quo, the first real-world evidence that RMAB-based targeting can outperform standard approaches [research.google](#) . It also handily beat the round-robin calls (which showed only a ~5% improvement, not significant) [research.google](#) . **Figure 1** below illustrates the cumulative impact: the RMAB group (blue) prevented far more drop-offs each week than the round-robin group (orange).

Figure 1: Cumulative engagement drops prevented by week, comparing the RMAB policy vs. a round-robin call policy in the ARMMAN field study. The RMAB strategy consistently outperformed the naive baseline, resulting in a 32% reduction in total drop-offs by week 7 [research.google](#) [research.google](#) .



This study, published at AAAI 2022, demonstrated the feasibility and impact of RMABs for maternal-child health outreach [research.google](#) [research.google](#) . The system was handed off to the NGO for real use after the trial [research.google](#) . Notably, the researchers addressed ethical concerns: an NGO ethics board approved the study, **data remained with the NGO and was anonymized (no personally identifiable info used)** [research.google](#) , and sensitive attributes like caste or religion were not collected to avoid bias [research.google](#) . They monitored model fairness by checking that the intervention distribution across income and education levels was appropriate, and indeed the model slightly *avored outreach to lower-income, less-educated beneficiaries*, aligning help with greater need [research.google](#) . ARMMAN experts were closely involved throughout to ensure the system was sensible and acceptable [research.google](#) .

SAHELI Mobile Health RMAB Program (2023)

Building on the above success, the collaboration with ARMMAN scaled up into a deployed system called **SAHELI** (“friend” in Hindi). SAHELI is essentially the production version of the RMAB approach, extended to a larger population and integrated into ARMMAN’s operations. It is described in an upcoming IAAI 2023 paper as “*the first deployed application of RMABs in public health*”, now **in continuous use by ARMMAN’s team** [research.google](#) . By 2023, SAHELI had already reached ~100,000 beneficiaries and was on track to serve **1 million mothers and children** by the end of 2023 [research.google](#) [research.google](#) . This scale far exceeds the initial pilot, indicating a robust deployment.

Innovations & Design: Achieving this scale required several enhancements to the original model and pipeline. The team implemented “multiple innovations in the RMAB model and its development, in preparation of real world data, and in deployment practices” [research.google](#) . In practice, this meant:

- **Better model fitting:** Using the growing dataset of interactions to continually improve the estimates of transition probabilities and reward parameters. They likely experimented with different machine learning techniques to predict which beneficiaries would benefit from calls (possibly incorporating more features or improved clustering). The paper mentions performing model selection and real-world evaluation of various RMAB system variants.
- **Operational integration:** Deploying SAHELI on cloud infrastructure as a platform that ARMMAN staff can use. It schedules weekly call lists for health workers, handling data ingestion from ARMMAN’s database and outputting the prioritized beneficiaries for outreach. The system had to be reliable and easy to use in a low-resource setting.
- **Responsible AI practices:** Emphasis was placed on ethics and fairness in deployment. This includes respecting data privacy (the NGO retains data control, models use anonymized data similar to the pilot) and making sure the system is **scalable to new contexts** (the paper notes SAHELI was built to be a general platform that can be scaled to other NGOs or domains) [storage.googlea...](#) . The **source code was made available** (open-sourced on GitHub) to encourage adoption and transparency [storage.googlea...](#) [storage.googlea...](#) .

Outcomes & Context: While detailed performance results of SAHELI at scale are not yet published in the abstract, the fact that ARMMAN expanded it to 100K+ users suggests sustained positive impact. This real-world rollout highlighted challenges like data quality (e.g. handling missing or delayed data on who listened to messages), user adoption by health workers, and maintaining engagement over longer periods. Overcoming these issues in SAHELI provides valuable lessons for Suvita. For example, ARMMAN’s focus was on maximizing listening to health info, whereas **Suvita’s end-goal is actual vaccinations**, which are more sparsely timed events. SAHELI’s success also shows the importance of an end-to-end pipeline: from predicting risk, to scheduling interventions, to monitoring results, all integrated with the NGO’s workflow.

Comparison: Suvita vs. ARMMAN/SAHELI Settings

There are strong parallels between Suvita’s scenario and the ARMMAN case, but also important differences in context and constraints:

- **Population Scale & Data:** Suvita’s potential beneficiary pool is extremely large – on the order of millions of infants annually (all of Maharashtra and Bihar’s newborns) [givewell.org](#) . ARMMAN’s pilot involved ~23k and their scale-up ~100k beneficiaries. Suvita will need to handle a wider geographic scope and data from government sources. While ARMMAN could directly record engagement (via automated call logs), Suvita relies on government health records (e.g. the RCH database) to track vaccinations. These records may be delayed or incomplete, making reward feedback noisy. Data issues like **missing phone numbers or outdated records** are a known concern (Suvita is actively monitoring if they “accurately collect phone numbers” and if SMS reach the phones) [givewell.org](#) . Suvita will likely need robust data cleaning and perhaps proxy metrics (like whether an immunization happened at the clinic) to infer states.
- **Outreach Modality:** ARMMAN’s intervention was a **direct phone call from a health worker to the beneficiary** – a one-to-one interaction. Suvita’s model includes **community ambassadors** as intermediaries: a volunteer might remind several families in their village. This is a more diffused intervention; its effectiveness can vary by the ambassador’s motivation and influence. It’s not as guaranteed that a given caregiver actually receives the message when the action is “prompt the ambassador.” In RMAB terms, the action effect for Suvita might be probabilistic in a different way (it depends on the ambassador’s follow-through). Suvita might consider direct calls or home visits for the highest-risk cases, but currently it primarily leverages the volunteer network. Thus, the “action” in Suvita’s RMAB could be *assigning a staff follow-up or intensified ambassador outreach* to a caregiver, whereas in ARMMAN it was always a staff call. Suvita’s outreach channels (SMS + peer reminder) are cheaper but possibly less immediately impactful per attempt than a personalized call – the model will need to account for that difference.
- **Reward Definition:** In Suvita’s case, the reward is tied to health behaviors like vaccination **completion** (a relatively infrequent but critical binary event for each vaccine due). In ARMMAN, reward was tied to weekly engagement (a more continuous behavior). This means Suvita’s reward signal is sparser (only when a vaccination session comes due) and perhaps more delayed (you might not know if the caregiver acted on the reminder until the clinic data is updated). The RMAB state for a caregiver might need to represent “on-track vs. off-track for the next vaccine,” which can be more complex than ARMMAN’s last-week engagement state. Also, the stakes for missing one intervention are higher (missing a vaccine has health consequences), so the policy objective might prioritize *minimizing missed vaccines* rather than just maximizing count – a subtle shift in reward structure that could require custom weighting.

- **Non-stationarity:** Both settings face non-stationarity, but of different kinds. In ARMMAN, a woman’s likelihood to listen may decrease over time as novelty wears off, etc., which the two-state Markov model attempts to capture. In Suvita’s immunization timeline, non-stationarity is inherent: as the child ages, the context changes (early infant vaccines vs. toddler boosters are different). After a child finishes their vaccination schedule (say by 15 months), they “exit” the program. New mothers are continually entering. This means the population is more dynamic, and the transition probabilities might not be constant over the whole timeline. Suvita may need to treat each vaccine stage as a separate phase or use a **semi-Markov** model where transitions depend on time-since-last-vaccine (for instance, the probability of eventually getting a 10-week vaccine if missed at 10 weeks might decay as weeks pass). The ARMMAN model assumes a homogeneous weekly transition, which might not hold for immunization schedules without adjustment.
- **Contextual Features:** Suvita can leverage **rich contextual data** from health records and demographics. For example, they might know the village or clinic each caregiver is associated with, whether an ambassador is assigned to that village, the mother’s education level, etc. These features could inform the likelihood of vaccination (e.g. certain districts or communities may have consistently lower uptake, indicating more effort needed). ARMMAN’s study did use demographics in clustering (age, income, number of children, etc.) [research.google](#) , but Suvita’s context might bring in different features like geographic access or whether the family has to travel far for a clinic. Also, ARMMAN explicitly chose not to collect sensitive data like caste/religion [research.google](#) ; Suvita likewise avoids that, but could use other proxies for vulnerability (as ARMMAN did with income and education [research.google](#)). In summary, Suvita’s RMAB solution may need to be more **context-aware** (a step toward *contextual RMAB*), treating arms with different profiles appropriately.
- **Existing Infrastructure:** Suvita partners with government systems, meaning it has some infrastructure in place (e.g. a Cloud SQL database storing beneficiary info from the RCH system, perhaps integration with government SMS gateways). This can be an advantage (access to a broad dataset) but also a constraint – the solution must integrate with existing IT and comply with government data policies. ARMMAN, as an NGO, had more control over their data pipeline but on a smaller scale. Suvita might also have to operate under stricter cost constraints (since it’s scaling a proven intervention at low cost). Fortunately, both SMS and ambassador approaches are extremely low-cost per additional child reached [founderspledge....](#) , so an RMAB system that boosts their effectiveness could be very cost-effective. The key is ensuring the system slots into Suvita’s current operations: the staff should be able to use it without heavy training, and it should not disrupt relationships with government partners or volunteers.

In summary, Suvita’s problem setting diverges from ARMMAN’s in scope and complexity – larger scale, volunteer-driven outreach, and discrete health outcomes – but the core challenge is analogous: *how to target limited person-to-person outreach to maximize health impact*. Insights from the ARMMAN/SAHELI studies (e.g. the clustering approach for learning, the success of Whittle index policies, and the importance of ethical deployment) provide a valuable starting point for Suvita’s RMAB solution, with appropriate customization to its context.

3. PoC Technical Blueprint

To validate the RMAB approach for Suvita, we propose building a proof-of-concept (PoC) in a Google Colab notebook using **synthetic data**. This prototype will demonstrate how an RMAB agent could learn and make decisions in an environment similar to Suvita’s, without needing real personal data. The PoC will cover data simulation, model inference, and decision recommendation, showing end-to-end functionality. Below is a blueprint of the components:

A. Simulating Caregiver Data: We generate N synthetic caregiver “arms” with simple two-state dynamics:

- Each arm has an internal state $s_t \in \{0, 1\}$ (0 = inactive/unvaccinated, 1 = active/vaccinated on schedule). The state evolves weekly or at each decision step.
- We define **transition probabilities** for each arm based on a *two-state Markov model*. For example, an arm i might have:
 - If **no action**: $P(s_{t+1}=1 \mid s_t=1) = p_{\text{stay}}$ (stay engaged), and $P(s_{t+1}=1 \mid s_t=0) = q_{\text{spont}}$ (spontaneous transition to engaged). This represents the “passive” dynamics.
 - If **action taken** (extra outreach): $P(s_{t+1}=1 \mid s_t=1) = p'_{\text{stay}}$ (maybe similar or slightly boosted), and $P(s_{t+1}=1 \mid s_t=0) = q'_{\text{spont}}$, where typically $q' > q$ (the intervention increases the chance of moving to the active state).
- We sample these probabilities for each arm from some distribution to introduce heterogeneity. For instance, we might say most caregivers have a 20% chance to self-activate ($q \sim 0.2$) and a 50% chance if nudged ($q' \sim 0.5$), but some “harder” cases have lower probabilities, etc.
- The **reward** at each step can be defined as the child getting vaccinated in that period. We can tie reward to reaching state 1 (e.g., reward = 1 if the caregiver is active at $t+1$, meaning they did vaccinate on time, otherwise 0). Alternatively, reward could be defined as the **increase in engagement** achieved (for simplicity, using the state as a proxy is fine).

Using these dynamics, we simulate a time series of say 50 weeks for all arms. We can simulate an *exploration policy* (e.g. randomly choose which arms to intervene on each week) to generate an interaction log. Each log entry might look like: `(arm_id, week, state_before, action, state_after, reward)` .

B. Bayesian Inference with `bayesianbandits.Agent` : We will employ a Bayesian learning approach to infer the model parameters from the simulated log. The idea is to mimic how Suvita's system would learn unknown engagement dynamics over time.

- Each arm's unknown transition probabilities (q , q' , p , p') can be given prior distributions. Since the reward/outcome is binary (vaccinated or not), a **Beta-Bernoulli model** is appropriate: we can maintain Beta prior distributions for the success probabilities of each arm under each action. For example, for the “inactive $\rightarrow$ active if acted upon” probability q' , we start with a Beta(1,1) (uniform) prior and update it with each observed outcome of an intervention on that arm.
- The `bayesianbandits.Agent` (a hypothetical class in our prototype) will encapsulate this learning. It takes in new observations (via an `/infer` step) to update priors/posteriors for each arm's probabilities. Essentially, it performs Thompson sampling-style updates or Bayesian belief updates for the transition model of each arm. Over time, the agent becomes more confident about which arms have high uplift from intervention.
- We will demonstrate this by feeding the simulated data into the agent. For example, we simulate week by week, and at each week, we call `agent.update(arm_id, action, reward)` for each interaction. We can then inspect the agent's learned parameters to see if they converge near the true values (for synthetic data we know the ground truth).

C. Computing Whittle Indices & Recommending Actions: With an updated belief model, the agent can compute a **Whittle index** for each arm at the current time. The Whittle index is a numeric priority score indicating how valuable it would be to intervene on that arm now [research.google](#) . In our two-state setting, the Whittle index can be derived analytically from the believed transition probabilities and reward structure. Intuitively, an arm that is currently in a “bad” state and that the model believes has a high chance to be rescued by an action (high q' relative to q) will have a high index (high potential gain). An arm that is doing well or unlikely to improve with help will have a lower index.

- The prototype will implement an `agent.recommend(k)` method that returns the top `k` arms to act on, based on current indices. For example, each round we might pick $k = 10$ arms out of N that have the highest indices as the recommended outreach list.
- We will illustrate how this recommendation changes over time. Initially, if the agent is uncertain, it may recommend arms randomly (or based on priors). As it learns, we expect to see it focus on the arms that truly benefit from interventions (in the synthetic data, those with big $q' - q$ gaps).
- The Colab can plot the **evolution of indices** for a few sample arms across weeks, showing how an arm that consistently doesn't respond to interventions will see its index drop, whereas a responsive arm's index stays high whenever it falls into the risky (inactive) state. We will also plot **cumulative reward (vaccinations) achieved** by following the agent's policy versus a baseline (random allocation or round-robin). This will showcase the reduced regret when using the RMAB strategy.

D. Required Libraries and Setup: The notebook will utilize open-source Python libraries:

- `numpy` and `pandas` for data manipulation and simulation.
- `random` or `numpy.random` for stochastic simulations.
- A custom or simple library for bandits – we assume a `bayesianbandits` module (if not already available, we can write simple functions for Beta-Bernoulli updates and Whittle index computation ourselves).
- `matplotlib` or `seaborn` for plotting results (trends of engagement, index values, regret curves).

The pseudocode for the main loop might look like:

```
agent = bayesianbandits.Agent(num_arms=N, prior_params={...}) env =
SimulatedEnvironment(arms=generated_arms) for week in range(50): # agent decides on top
K arms to intervene target_arms = agent.recommend(k=K) outcomes = [] for arm in
range(N): action = 1 if arm in target_arms else 0 reward = env.step(arm, action) #
advance one time step for that arm outcomes.append((arm, action, reward))
agent.update_batch(outcomes)
```

We will verify that this loop runs and produces reasonable output. The expected outputs include:

- **Learning curves:** e.g. a plot of the agent's belief (posterior Beta mean) about a particular arm's “success probability if called” over time as it observes more data.
- **Policy performance:** a plot of cumulative reward of the agent's policy vs a naive policy, demonstrating improvement. We expect to see the RMAB agent achieving higher vaccination rates over time by focusing on the right arms.
- **Sample decision snapshot:** printing the top recommended arms at a given week and their indices, to qualitatively show that it's picking, say, caregivers who have missed last vaccine but have a good chance to catch up if nudged.

Through this synthetic PoC, the team can build intuition and confidence in the RMAB approach, setting the stage for real-data integration.

4. MVP Design & Custom Deployment

With a working prototype, the next step is to integrate the RMAB solution into Suvita's existing tech stack as a **Minimum Viable Product (MVP)**. Suvita's infrastructure likely includes a centralized database of beneficiaries (perhaps a Cloud SQL instance where enrollment data from the government's RCH system is stored), messaging systems for SMS and voice calls, and maybe event streams for when births are registered or vaccines are given. We propose a lightweight, **cloud-based deployment** that augments this infrastructure with intelligent decision-making while keeping operations simple.

Architecture Overview: The MVP system will consist of a **Bandit Decision Service** and a data pipeline connecting it with Suvita's databases and messaging channels:

- The **Bandit Service** (let's call it `rmab-service`) will be a stateless microservice accessible via API endpoints. We can deploy this on **Cloud Run** (a serverless container platform on GCP) to automatically handle scalability and only incur costs when requests come in. This service encapsulates the core logic: maintaining the bandit model (or loading the latest learned model) and computing recommendations.
- A **shared storage** (Cloud SQL or Cloud Firestore) will hold the model's persistent data – for example, the current state and parameter estimates for each caregiver arm. This could also be stored in a fast key-value store like **Redis** (which Suvita might already use for caching SMS queues, etc.), but using the existing Cloud SQL is simplest. The table might look like `CaregiverState( id, last_state, last_vaccine_date, posterior_params )` to keep track of each arm's info.
- **Data input pipeline:** Whenever relevant events occur (new enrollment, a vaccination done, a missed appointment, etc.), the system should update the bandit model. Suvita can use **Cloud Pub/Sub** to publish events (for example, “Child X missed their 6-week vaccine” or “Ambassador Y reported family Z got vaccinated”). A small **Cloud Function** can subscribe to these events and call the Bandit Service's `/infer` endpoint with the new data, triggering an update to that arm's state. If Pub/Sub is not in use, even a direct API call or a cron job that periodically reads the database for changes can work.
- **Recommendation output pipeline:** On a regular schedule (say weekly, or daily), the system should decide which caregivers to target with extra outreach. Using **Cloud Scheduler**, we can trigger a Cloud Run execution or Cloud Function that calls the Bandit Service's `/recommend` endpoint. This endpoint will return a ranked list of, say, `K` caregivers who are highest priority for follow-up before the next clinic day. The result can then be written to a table (e.g. `OutreachList(date, caregiver_id, recommended_action)`) and/or sent as a Pub/Sub event that notifies another process to act (for example, an ambassador coordinator module or an automated dialer for phone calls).

API Endpoints Design: The Bandit Service will expose simple RESTful endpoints:

- `POST /infer` – Accepts new observation data for one or multiple arms. For example, Suvita's app could POST a JSON: `{"caregiver_id": 123, "observed_reward": 0, "action": 0}` meaning caregiver 123 did **not** vaccinate (reward 0) when no extra action was taken this round. The service will update the model's belief for that caregiver (e.g. adjust the Beta posterior for that arm). This can be batch-called for efficiency (multiple observations at once).
- `GET /recommend?count=K` – Triggers the computation of top-K Whittle indices and returns the recommended caregiver IDs (and perhaps an estimated confidence or score). Suvita can specify how many they can follow up on (e.g. `K=100` for a week given staff/volunteer capacity). Optionally, we can have different recommendation types: e.g. one list for “which ambassadors to call and motivate” and another for “which caregivers to directly call”, if Suvita decides to have a small call center for critical cases.
- (Optional) `POST /reset` or `/train` – to reinitialize or retrain the model from scratch on historical data. This could be used if we periodically retrain the model in batch (say once a month, incorporate all new data and re-cluster or re-fit parameters), rather than purely online updates. This retraining could be done offline in a Colab or a scheduled job and the new model parameters uploaded to the service.

Integration with Suvita's Ops: Once the Bandit Service provides a list of target caregivers, Suvita can implement the outreach using its existing channels. For example, suppose on Monday the system outputs 100 caregivers across Maharashtra whose children are due for shots but likely to miss without intervention. These could be fed into:

- An automated **call campaign**: use a Twilio-like service or the government's call system to deliver a personalized reminder call to those caregivers, or connect them to a helpdesk.
- or, the **ambassador network**: the system knows which ambassador is linked to each caregiver (since Suvita has that mapping). It can send a message to the *specific ambassadors* responsible for those high-priority caregivers: e.g. “Reminder: please check on [Mother A] in your village this week, as her baby is due for vaccination.” This way, volunteer effort is directed to where it's needed most. (This essentially treats ambassadors as an extension of the “action.”)

Because Suvita's staff is lean, the deployment emphasizes **serverless and managed services** to minimize maintenance:

- **Cloud Run** will keep the model service highly available without anyone managing servers.
- **Cloud Functions + Pub/Sub** handle event-driven updates and scheduled triggers without a dedicated server polling.
- The system can reuse Suvita's existing Cloud SQL for storage, avoiding introducing new databases. If high-speed caching is needed, a managed Redis (MemoryStore) could be used for quick lookups of arm states.
- For periodic heavier computation (like re-estimating model clusters or training a more complex model on new data), Suvita can use a **preemptible VM or Cloud Dataproc** on a schedule. For instance, a weekly job that runs a Python script to retrain transition probabilities using all data (similar to Google's approach of clustering offline) could be scheduled. Using a preemptible VM instance keeps costs low, and since it's not user-facing it can handle interruptions or retries.

Alternate Deployment Options: While GCP is convenient (especially if Suvita already uses it for their database and messaging), the architecture can be replicated on other platforms or on-premise:

- **AWS:** Use AWS Lambda for the functions (or Fargate for the service), SNS/SQS in place of Pub/Sub for event messaging, and RDS instead of Cloud SQL. AWS EventBridge could schedule the recommendation calls similarly.
- **Azure:** Azure Functions and Azure Container Instances could serve the model; Service Bus or Event Grid for messaging.
- **On-Premise:** If needed, the entire system could be dockerized and run on a small server at Suvita. A cron job could replace Cloud Scheduler, and a local RabbitMQ or Kafka could substitute Pub/Sub. However, on-prem would require more IT support and is less desirable for a small team.

The MVP should be implemented in a modular, configurable way. In particular, thresholds like how many caregivers to target, or the minimum confidence to intervene, should be tunable by Suvita’s program managers. The system’s recommendations will initially be used in a **decision support** role – i.e. suggesting whom to target, but Suvita’s team will monitor and provide feedback. Over time, as trust in the model grows, it can be given more automated control of outreach allocation.

In summary, the deployment plan integrates the RMAB intelligence into Suvita’s workflows with minimal disruption. By leveraging cloud serverless tech, we achieve a low-ops solution: essentially **“plug in” a smart brain to Suvita’s existing body** of data collection and messaging, thereby empowering the team to do more with their limited resources.

5. Advanced Roadmap

Once the MVP is running and demonstrating impact (even on a pilot basis), there are several enhancements and extensions to explore. These will improve the model’s fidelity, adapt it to changing conditions, and make it easier to scale and share with others:

- **Contextual RMAB:** Incorporate richer caregiver and context features into the bandit’s decisions. In the current model, arms might be treated as homogeneous aside from their learned state. In reality, different communities or individuals may respond differently to interventions. A contextual RMAB would use side information (e.g. the presence of an ambassador in the village, urban vs rural location, socio-economic indicators, etc.) to modulate the policy. Practically, this could mean maintaining separate models for different contexts or adding context vectors into the state. Recent research on **Bayesian contextual RMABs** provides methods to do this efficiently (e.g. using Thompson sampling with context to learn policy – see BCoR, 2022). For Suvita, a contextual approach could answer questions like “Caregivers who live in very remote areas have lower baseline vaccination rates – should we always prioritize them for calls?” and adjust the indices accordingly to ensure equity and effectiveness.
- **Non-Stationary Dynamics & Semi-Markov Extensions:** Over a multi-year horizon, the assumptions of a fixed transition probability may break. Behavior can shift due to external factors (e.g. COVID pandemic disruptions, seasonality in disease outbreaks, or program changes). We should plan to detect and adapt to non-stationarity. Techniques include **sliding window learning** (giving more weight to recent data) or **change-point detection** (resetting the model if a significant behavior change is detected). A semi-Markov model could be introduced where the time since last vaccination or last intervention is part of the state, allowing the model to represent, for instance, that the longer a child has been overdue, the harder it may be to get them back (or conversely, urgency increases). These enhancements will make the model more robust over time and prevent degradation if initial assumptions become outdated.
- **Inverse Reinforcement Learning (IRL) and Decision-Focused Learning:** The current approach assumes we know the reward function (e.g. each on-time vaccination yields the same reward). However, health experts might have higher-level goals, like “prioritize the most vulnerable populations” or “ensure no child misses more than one vaccine.” IRL can be used to **learn the reward or priority structure from expert behavior or preferences**. A recent paper by Jain et al. (2024) demonstrates the first use of IRL for RMABs in maternal health, showing improved outcomes by learning an objective aligned with expert goals [arxiv.org](#) . We could allow experts to input some desired criteria (aggregate targets or fairness constraints), and then use an algorithm like WHIRL (as in that paper) to adjust the indices in a way that meets those criteria [arxiv.org](#) . Another angle is *end-to-end decision-focused learning*: instead of just plugging in estimated probabilities into a fixed index formula, we could directly train a neural network to output an index or action that maximizes long-term reward on historical data. This could potentially capture complex patterns that a simple model misses. Such approaches would be experimental but could yield a policy that’s better tailored to Suvita’s true objectives.
- **Open-Sourcing and Community Build-out:** Following the example of ARMMAN’s SAHELI (which has code available on GitHub [storage.googleapis.com/kestra-ai/SAHELI](#)), Suvita should plan to open-source the core components of this system. By publishing the code, along with documentation and case studies, we enable other NGOs or health departments to replicate and benefit from these tools. To facilitate easy adoption, we can create **Helm charts** or **Terraform modules** for the deployment, allowing an organization to spin up the whole pipeline on their cloud with minimal configuration. Additionally, provide sample synthetic data and a tutorial (perhaps via the Colab notebook) so others can learn how to adapt the model to their needs. Over time, we envision a **community of practice** around RMAB for social impact – NGO staff, researchers, and developers sharing improvements, new features (for example, someone might contribute a new algorithm for contextual bandits, or integration with a new SMS gateway), and supporting each other. Suvita can collaborate with research groups (like Google AI for Social Good or university labs) to host workshops or publish results, further solidifying the community. This roadmap item isn’t just technical; it’s about scaling impact through knowledge transfer.

- **Additional Health Domains:** Though not a core part of the maternal-child immunization scope, the system could be extended to other health behaviors in the future (given its general nature). For instance, reminders for antenatal care visits (which Suvita already does via SMS) could be optimized similarly, or even beyond maternal health – e.g., TB treatment adherence support, or nutrition counseling follow-ups. The architecture is generic; adding new arms or new reward definitions is mainly a matter of plugging in the data. This could position Suvita as a leader in deploying AI for various health outreach programs.

Each of these enhancements should be approached in phases – tested in simulation or small pilots, evaluated for improvement, and then integrated. By iterating in this way, Suvita can maintain a stable production system while continuously incorporating cutting-edge ideas that amplify its effectiveness and reach.

6. Risks & Mitigations

Implementing an RMAB-driven outreach system comes with both technical and social risks. Here we identify major risks and strategies to mitigate them:

- **Model Misspecification or Algorithmic Error:** If the transition dynamics or reward models are mis-modeled (e.g. assuming independence when peer effects exist, or setting wrong reward weights), the recommendations might be suboptimal or even counterproductive. **Mitigations:** Start with pilot experiments and A/B tests – for example, compare the RMAB-selected group vs a control group in a small region to verify improved outcomes before scaling up. Incorporate domain knowledge: have Suvita’s field experts review the list of recommended high-priority cases to see if they make intuitive sense, and adjust model features if not. Continuously monitor key metrics (like vaccination rates, or any signs of decreased coverage in any segment) to catch if the model is driving unexpected results. If needed, maintain a fallback policy (like random or rule-based targeting) that can be used if the system needs to be paused for investigation.
- **Data Sparsity and Quality:** The model's learning is only as good as the data it receives. Infrequent events (e.g. a rare vaccine schedule) might not provide enough feedback to learn effectively. Data could also be incomplete (missing records of a vaccination) leading the model to think a caregiver is still “inactive” when in fact the child got vaccinated elsewhere. **Mitigations:** Augment the bandit with **offline learning** on any available historical data (even if sparse) to set priors. Use clustering or hierarchical models to share strength between arms, as done in the ARMMAN study [research.google](#) – this way even new or rarely-seen scenarios can be handled by borrowing information from similar cases. On the data engineering side, implement validation checks: for instance, cross-verify SMS delivery receipts, periodically survey a sample of caregivers to ensure the records reflect reality (did they get the SMS? did they vaccinate?). The monitoring questions Suvita already plans (like “do participants say the messages made a difference?” [givewell.org](#)) will provide qualitative signals if the data pipeline misses something. In summary, use conservative assumptions when data is uncertain (don't confidently skip someone if you aren't sure they actually got vaccinated) and improve data collection over time (perhaps integrate with government immunization tracking systems for real-time updates).
- **Compute and Scalability Limits:** An RMAB algorithm, especially with many arms and complex contexts, could become computationally intensive. Calculating Whittle indices for 1 million arms in real-time might strain servers, and retraining models on large data might be slow. **Mitigations:** Leverage the **cluster-level decision** approach – instead of computing an index for each of a million individuals separately, group similar individuals and compute indices per group [research.google](#) . This dramatically cuts down computation and was shown to work well in practice. Also use scalable cloud resources: the serverless approach will automatically allocate more CPU when needed. If using a batch job for retraining, run it on a powerful machine (even temporarily) or use vectorized computations (NumPy/Pandas or even GPU if needed). In terms of software, the algorithms can be optimized in C++ or PyTorch if Python loops become a bottleneck. Since interventions are planned e.g. weekly, we have the advantage of time – even if it takes a few hours to crunch numbers for a million records, that’s acceptable as a nightly job. We will also prioritize maintaining a **lightweight model** (e.g. a two-state bandit is quite simple) unless evidence shows a need for more complexity.
- **User Privacy and Security:** Handling personal health data (phone numbers, vaccination records) demands strict privacy protection. There is risk of data leaks or misuse of sensitive information. **Mitigations:** Follow the principle that **all personal data stays with Suvita/partners, not the algorithm**. The Bandit Service should use anonymized IDs for caregivers – no phone numbers or names should flow through the model, only an internal ID and features like “rural vs urban” or age group, which aren't personally identifying on their own [research.google](#) . Ensure all databases are secure (encrypted at rest and in transit). We will enforce access controls so that only authorized staff or services can call the RMAB endpoints (e.g. require authentication tokens). If open-sourcing, we'll provide dummy data configurations, and make sure no actual private data is in code repos. Additionally, since the system may nudge outreach to certain individuals, we must ensure it doesn't inadvertently expose any private status. For example, an ambassador should not be told *why* a certain caregiver is high priority (to avoid revealing health info), only that they should check in on them. Essentially, we use the model's outputs internally to guide actions without publicly labeling someone as “high risk.”

- **Bias and Fairness Concerns:** Algorithms can reinforce biases – e.g. if certain communities historically have lower uptake (perhaps due to access issues), the model might keep focusing on them and possibly *overlooking others who also need help*. Or it might allocate resources away from communities that always comply (since they don't “need” intervention, but we must ensure they don't feel neglected by the program). **Mitigations:** Bake in fairness checks and constraints. We will regularly analyze the distribution of who gets extra outreach versus the overall population (as the Google team did, checking income/education breakdown [research.google](#)). If we see systematic disparities that are not intended (say, the model never gives a certain district any calls), we investigate why. It could be the model learned they always vaccinate without help – which is fine – or it could be a data issue. Suvita might set simple rules to ensure equity, for instance: guarantee that every district gets at least some ambassadors/calls each cycle, or cap the number of times the same family is pinged (to avoid annoyance). Involving community representatives in reviewing the strategy can also help catch bias – e.g. get feedback from health workers in different regions about whether the suggestions seem fair and appropriate.
- **NGO Capacity and Misuse:** A sophisticated AI system can be intimidating or misused if the team isn't fully on board. If Suvita’s staff find it too complex, they might revert to simpler methods, or conversely they might over-rely on it without understanding limitations. Also, as this might be one of the first RMAB deployments for immunization, there is the broader risk of misuse by others who copy it without proper care (hence causing harm or disillusionment). **Mitigations:** Invest in **training and documentation** for Suvita’s team – demystify how the system works, perhaps through the PoC notebook and visualizations. Make the interface simple (clear lists of names to follow up, not a black-box score). Start by using the model's output to assist (a recommendation list) rather than fully automate actions, so staff can provide oversight. Collect qualitative feedback from ambassadors and beneficiaries: did the outreach frequency feel right? Any complaints? This human-in-the-loop approach will catch issues early. To prevent misuse externally, when open-sourcing, include guidance and disclaimers: emphasize the importance of local customization, ethical checks, and not using the tool in contexts it wasn't designed for without expert consultation. Possibly form a **governance group** with members from different NGOs and maybe academic advisors to steer the development of the project as it grows – this can ensure that one organization isn't pushing changes that could negatively affect others, and that there is accountability in how the AI is used for social impact.

In conclusion, while there are real risks in deploying an AI-driven outreach system, each is addressable with careful planning and oversight. By taking a responsible innovation approach – testing thoroughly, involving stakeholders, and gradually ramping up – Suvita can safely harness the power of RMABs to improve maternal and child health outcomes at scale. The potential reward (significantly higher vaccination rates and lives saved) is well worth the effort of navigating these challenges.

Sources:

1. Google Research & ARMMAN's RMAB field study (Aditya Mate et al., AAAI 2022) – demonstrated a 30% boost in engagement by prioritizing calls to at-risk mothers [research.google](#) [research.google](#) .
2. SAHELI deployment (IAAI 2023) – first large-scale RMAB application in health, reaching 100k+ beneficiaries in India [research.google](#) [research.google](#) .
3. Suvita program description (GiveWell, 2024) – outlines Suvita's SMS reminders and ambassador model, and expansion plans across Bihar & Maharashtra [givewell.org](#) [givewell.org](#) .
4. Suvita’s context (Founders Pledge, 2022) – highlights low vaccine coverage in India and the use of local ambassadors identified as opinion leaders [founderspledge....](#) [founderspledge....](#) .
5. ARMMAN blog (Google AI, 2022) – details the RMAB modeling (two-state MDP per beneficiary) and the clustering + Whittle index approach for solving it [research.google](#) [research.google](#) . Also discusses ethical safeguards like data anonymity and fairness checks [research.google](#) [research.google](#) .
6. Jain et al. “IRL for Restless Bandits” (arXiv 2024) – introduces learning reward preferences (WHIRL) for RMABs in health, enabling alignment with expert goals and improved outcomes [arxiv.org](#) .



I think there were some blindspots in your design, what about this design

Suvita RMAB: Optimizing Caregiver Engagement for Vaccination Uptake

1. Introduction & Problem Statement

Objective: To significantly improve caregiver engagement and vaccination attendance in Suvita's mHealth programs by intelligently allocating limited intervention resources (SMS, Health Worker Calls) using a Restless Multi-Armed Bandit (RMAB) framework.
Challenge: Caregiver engagement with automated health messages (SMS) tends to decline over time. Suvita's health worker resources for personalized follow-up calls are limited. This project aims to prioritize interventions for caregivers who will benefit

most, maximizing vaccination uptake.

Inspiration & Leverage: This design heavily draws upon the successful implementation and learnings from the SAHELI project (Verma et al., Mate et al.), which deployed RMABs for maternal and child care with NGO ARMMAN, and utilizes their open-source codebase as a foundational reference, particularly for Whittle Index computation.

2. Core RMAB Approach (Vanilla RMAB)

Modeling Caregivers: Each caregiver's engagement journey is modeled as an individual arm in an RMAB system. The underlying model for each arm is a Markov Decision Process (MDP) with states representing different levels of engagement.

States (Initial MVP): Two primary states based on SMS interaction:

- S_Responsive: Caregiver has recently interacted positively with an SMS (e.g., opened, clicked a link within the last X days).
- S_Unresponsive: Caregiver has not interacted positively with SMS recently.

Future: Explore more granular states (e.g., 'Lapsing', 'Highly_Responsive') if beneficial.

Actions (Initial MVP):

- A_Intervene: Send a targeted SMS or schedule a Health Worker call (initially, RMAB decides who to target; a simpler secondary logic might pick SMS vs. call).
- A_Passive: No direct intervention this cycle.

Rewards: The primary reward R is +1 if a vaccination visit occurs within a defined window following an intervention or in a given cycle, 0 otherwise. Small negative costs can be associated with A_Intervene.

Restlessness: Transition probabilities $P(S'|S,A)$ and reward probabilities $P(R|S,A)$ are dynamic and learned over time.

Whittle Index Prioritization: The Whittle Index will be computed for each caregiver based on their current state and their cluster's learned MDP parameters. This index quantifies the marginal benefit of intervening. Caregivers with the highest indices will be prioritized for intervention, up to Suvita's budget (K interventions per cycle).

Whittle Index Solver: Adapt the binary search and value iteration algorithm from SAHELI's whittle_utils.py (planinf function) for Suvita's state/action/reward structure.

Sleeping State for Intervention Frequency: To avoid over-contacting, incorporate a "sleeping state" mechanism similar to SAHELI's. If an intervention occurs, the caregiver enters a temporary non-intervention period. SAHELI's $\eta=+\infty$ approximation (reducing the augmented MDP to ~4 core states for index computation) will be adopted for computational efficiency.

3. Phase 1: Proof-of-Concept (PoC) - Core Logic Validation

Objective: Validate the RMAB mechanism, parameter learning with bayesianbandits, and Whittle Index computation using synthetic data.

Tech Stack: Google Colab, Python, NumPy, bayesianbandits.

Components:

- Synthetic Data Generation: Simulate N caregivers with 2-state MDPs (e.g., S_Responsive, S_Unresponsive). Define $P(S'|S,A)$ and $P(R|S,A)$ (e.g., vaccination success). Generate a log of (caregiver_id, state, action, next_state, reward).
- Parameter Learning (bayesianbandits): Initially, treat each synthetic caregiver as an individual arm. Use bayesianbandits.DirichletClassifier (for Beta-Bernoulli parameters) to learn transition and reward probabilities from the synthetic log for each caregiver.
- Whittle Index Computation: Implement/adapt the planinf logic (from SAHELI reference) to take the learned parameters from bayesianbandits and compute Whittle Indices.
- Recommendation Simulation: Simulate a /recommend function: given current states, compute indices, select top-m.
- Evaluation: Plot simulated vaccination uptake vs. random/round-robin.

Deliverable: Documented Colab notebook demonstrating end-to-end flow.

4. Phase 2: Minimum Viable Product (MVP) - Real-World Integration & Initial Impact

Objective: Deploy a vanilla RMAB system using Suvita's real data, leveraging insights from SAHELI's pipeline for clustering and frugal operations.

Tech Stack: Python, FastAPI, bayesianbandits, adapted SAHELI Whittle solver, PostgreSQL (state/history/params), Redis (cache for current indices/states), Docker, Cloud Run (or similar serverless API deployment), Twilio.

Key SAHELI Adaptations for MVP:

Clustering for Parameter Learning:

- From Suvita's historical SMS data, define engagement states.
- Calculate historical passive transition probabilities for each caregiver.
- Use k-means (target k=20, tune via elbow method like SAHELI) on these passive transition vectors to create caregiver clusters.
- For each cluster, aggregate historical (S,A,S',R) data. Use bayesianbandits.DirichletClassifier to learn cluster-level $P(S'|S,A)$ and $P(R|S,A)$.

Features-Only (FO) Mapping (Cold Start): Train a simple classifier (e.g., RandomForest from scikit-learn) to map new caregivers' socio-demographic features (from Suvita's registry, guided by data_description.md from SAHELI) to one of the k clusters.

Components:

Data Ingestion & ETL:

Connect to Suvita logs (caregiver registry, SMS history, clinic attendance).

Lightweight Python ETL scripts.

Schema: PostgreSQL for caregiver static data, historical interactions, learned cluster MDP parameters. Redis for current caregiver states and their current Whittle Indices (or cluster mapping for on-the-fly computation).

RMAB Service (FastAPI):

/train_clusters (Periodic Batch Job):

Loads historical data.

Performs clustering based on passive transitions.

For each cluster, uses bayesianbandits to learn/update MDP parameters ($P(S'|S,A)$, $P(R|S,A)$).

Trains/retrains the FO mapping model (features -> cluster_id).

Persists cluster MDP parameters and FO model.

/precompute_whittle_indices (Periodic Batch Job, after /train_clusters):

For each (cluster_id, state_id) pair, uses the adapted planinf function and learned cluster MDP parameters to compute and store Whittle Indices (e.g., in PostgreSQL or Redis).

/assign_caregiver_cluster (On new caregiver registration or periodically):

Uses the FO mapping model to assign a cluster_id to new/unassigned caregivers.

/update_caregiver_state (Triggered by SMS interaction logs):

Updates the current engagement state of a caregiver in Redis/PostgreSQL.

/recommend?budget=<K> (Real-time API for intervention planning):

Fetches current state and cluster_id for all active caregivers.

Looks up the pre-computed Whittle Index for their (cluster_id, current_state).

Returns the top K caregiver_ids with the highest indices.

Outreach Integration:

/recommend output feeds into Twilio for SMS dispatch or updates ambassador call schedules.

Deployment: Dockerize FastAPI. Initial deployment on Google Cloud Run (following SAHELI's frugal approach).

Warmup Period: Implement a ~6-week warmup period for new beneficiaries (like SAHELI) before they are included in RMAB prioritization, allowing for initial data collection.

Monitoring & Initial A/B Test: Track intervention counts, SMS open/click rates, and (critically) vaccination uptake. Plan an A/B test: Group 1 (Current Suvita method) vs. Group 2 (RMAB-driven interventions).

5. Phase 3 & Beyond: Enhancements & Future Directions

Contextual Modeling:

Refine FO Mapping: Improve the features -> cluster mapping model.

Simple Contextual Bandits: Introduce MABWiser (e.g., LinUCB) to predict immediate SMS success (open/click) based on caregiver context, potentially as a layer before RMAB or to choose message variants.

Full Contextual RMAB: (Long-term) Explore models where caregiver context directly influences $P(S'|S,A)$ and $P(R|S,A)$ within the RMAB formulation for each cluster.

Evaluation:

Robust Off-Policy Evaluation (IPS, DR estimators) on logged data.

Continuous A/B testing for new features or model versions.

Scalability:

If planinf becomes a bottleneck (many clusters), explore JAX for XLA compilation and vectorization (vmap) of the Whittle Index computation, or parallelize per-cluster computations.

Optimize database queries and data pipelines.

Consider Kubernetes if Cloud Run limitations are hit at extreme scale.

Fairness & Responsibility:

Continuously monitor intervention distribution across demographic segments (guided by SAHELI's fairness analysis on education/income).

Implement mechanisms to mitigate identified biases if necessary.

6. Key Technical & Research Questions for Suvita:

What are the most discriminative engagement states for Suvita's SMS-based interactions?

How well do passive SMS engagement patterns cluster Suvita caregivers?

Which socio-demographic features are most predictive for mapping new caregivers to these clusters?

How sensitive is the system to the choice of gamma, k (number of clusters), and intervention budget?

Validating the impact on actual vaccination uptake (primary KPI).

Suvita Rmab Design

I've drafted the updated design doc, merging your detailed structure and the key SAHELI-inspired techniques into a clear, phased roadmap—from PoC to MVP and beyond. Let me know if you'd like further tweaks or deeper detail on any section!

what were the differences

Overview

The updated design incorporates a series of **cost-optimization** and **operational simplicity** enhancements that were not present in the original roadmap. While the core RMAB framework, PoC outline, MVP integration steps, and advanced roadmap remain, the revised design specifically:

- 1. **Emphasizes serverless compute** (Cloud Run, Functions) instead of always-on GKE clusters.
- 2. **Introduces preemptible/spot VMs** for batch retraining workloads.
- 3. **Reuses or right-sizes existing data stores** (shared Cloud SQL or Firestore free tier) rather than provisioning new instances.
- 4. **Optimizes the event bus** by batching Pub/Sub messages and sharing topics across projects.
- 5. **Leverages free-tier quotas and committed-use discounts** to further reduce monthly costs.

Below is a structured breakdown of those differences.

1. Compute Deployment

Aspect	Original Design	Updated (Optimized) Design
Interactive API Hosting	Always-on GKE cluster + node pools	Serverless via Cloud Run (scales to zero, pay-per-use)
Batch Jobs	GKE node pool with autoscaler	Cloud Functions or preemptible VM jobs for ETL/model retraining
Cost Control	Cluster management and VM fees	Eliminates cluster fees; uses free-tier vCPU/GiB-sec; spot VMs for heavy jobs

2. Data Storage & Reuse

Aspect	Original Design	Updated (Optimized) Design
Primary Database	Dedicated Cloud SQL instance	Reuse existing Cloud SQL (share with app)
Alternative Storage	Cloud SQL + Redis	Firestore (Datastore mode) for small state
Cache	Redis for state and index lookup	Right-size Redis or use Firestore free tier

3. Event Bus & Messaging

Aspect	Original Design	Updated (Optimized) Design
Event Bus	Pub/Sub or Kafka with dedicated topics	Share Pub/Sub topics across projects; cross-project IAM
Message Granularity	One event per action	Batch Pub/Sub messages (up to 100 ms delay per API call)

4. Cost-Saving Mechanisms

Aspect	Original Design	Updated (Optimized) Design
Free Tier Usage	Not explicitly leveraged	Cloud Run & Functions free quotas , Firestore free tier
Commitment Discounts	Not mentioned	Committed Use Discounts (1-3 year) on baseline CPU/Memory
Preemptible VMs	N/A	Use for non-critical batch retraining tasks to cut compute costs by 60–90%

5. Infrastructure as Code & Operations

- **Automation:** Both designs use Terraform/Helm for IaC, but the optimized design emphasizes **shared-infra modules** so NGOs can attach to existing networks/databases rather than recreating them.
- **Maintenance Overhead:** The optimized approach dramatically reduces ops burden by removing cluster management and encouraging serverless patterns.

Summary of Impact

By shifting from an always-on Kubernetes approach to a **serverless-first architecture**, reusing existing data stores, batching events, and tapping into free-tier/discounted pricing, the revised design can **reduce monthly costs by 50–70%** and lower operational complexity—making the RMAB solution far more attainable for Suvita and similar NGOs.

I'm planning to have a small video call with my friends to share my thoughts on all this. Help me make this message better. But don't change the style

Without any guardrails, the current economic and geopolitical incentives put us in a trajectory where human agency and participation gets eroded.I've been exploring certain solutions for mitigating these issues and would like to share it with a wider audience to seek feedback and eventually start doing something about it (building certain technical infrastructure and also some forms of advocacy).I'll be hosting a session on this on Wednesday (May 21) and if you find it exciting, you can play a role in shaping the future of humanity!

Here's your message enhanced slightly for readability and impact, without changing your personal style:

Without any guardrails, our current economic and geopolitical incentives push us towards a trajectory that steadily erodes human agency and participation. I've been exploring some potential solutions to mitigate these issues, and I'd love to share them with a wider audience, gather feedback, and potentially kick off some actionable next steps—like building technical infrastructure and initiating advocacy efforts.

I'll be hosting a session to discuss these ideas on Wednesday (May 21). If this resonates with you, join me—you might just help shape the future of humanity!

I'm planning to have a small video call with my friends to share my thoughts on all this. Help me make this message better. But don't change the style

Without any guardrails, the current economic and geopolitical incentives put us in a trajectory where human agency and participation gets eroded.I've been exploring certain solutions for mitigating these issues and would like to share it with a wider audience to seek feedback and eventually start doing something about it (building certain technical infrastructure and also some forms of advocacy).I'll be hosting a session on this on Wednesday (May 21) and if you find it exciting, you can play a role in shaping the future of humanity!

Without any guardrails, our current economic and geopolitical incentives push us towards a trajectory where human agency and participation gradually erode. I've been exploring potential solutions to these challenges and would love to share my thoughts with a broader group, gather feedback, and perhaps even kick-start actionable efforts—like building technical infrastructure or engaging in advocacy.

I'll be hosting a short session about this on Wednesday (May 21). If this sounds interesting to you, join in—you might help shape the future of humanity!

I want to mention that I'll be discussing my core ideas in depth and am looking for feedback and potential collaboration. Also did you think the last line is a bit grandiose

Here's a refined version maintaining your style, clarifying your intent, and softening the last line to sound less grandiose:

Without any guardrails, our current economic and geopolitical incentives push us towards a trajectory where human agency and participation gradually erode. I've been exploring some ideas and solutions to these challenges, and I'd like to discuss them in depth, gather your feedback, and explore potential collaboration—whether through building technical infrastructure, advocacy, or something entirely new.

I'll be hosting a short session about this on Wednesday (May 21). If you're interested, I'd love to have you there—your perspectives would be really valuable!